# The Concise TypeScript Book

Simone Poggiali

# The Concise TypeScript Book

The Concise TypeScript Book przedstawia kompleksowy i zwięzły przegląd możliwości TypeScript. Zawiera jasne objaśnienia wszystkich aspektów najnowszej wersji języka — od rozbudowanego systemu typów po zaawansowane funkcje.

Niezależnie od tego, czy jesteś osobą początkującą, czy doświadczonym programistą, ta książka stanowi nieocenione źródło wiedzy, które pozwoli Ci lepiej zrozumieć TypeScript i sprawniej się nim posługiwać.

Ta książka jest całkowicie bezpłatna i ma otwarty kod źródłowy.

Uważam, że wysokiej jakości edukacja techniczna powinna być dostępna dla każdego. Dlatego udostępniam książkę bezpłatnie i regularnie ją aktualizuję, wprowadzając ulepszenia i nowe przykłady.

Poznaj **The Concise TypeScript Book Plus Edition**.




Dla czytelników, którzy chcą wykroczyć poza wydanie open source, książka **The Concise TypeScript Book Plus Edition: React and Real-World Patterns for TypeScript 7** zawiera dodatkowe, ekskluzywne treści poświęcone praktycznym zastosowaniom.

Wydanie Plus obejmuje:

- **Aktualizację do TypeScript 7** — omówienie najnowszych funkcji i ulepszeń języka TypeScript 7.
- **TypeScript z Reactem** — praktyczne wskazówki dotyczące określania typów komponentów, propsów, hooków,

zdarzeń, elementów potomnych, referencji i typowych wzorców Reacta.
- **Przepisy TypeScript do rzeczywistych projektów** — konkretne przykłady dotyczące praktycznych problemów, z którymi programiści spotykają się podczas tworzenia i utrzymywania aplikacji TypeScript.

Kupując wydanie Plus, bezpośrednio wspierasz także dalszy rozwój i utrzymanie bezpłatnej książki o otwartym kodzie źródłowym.

Wydanie Plus jest dostępne w języku angielskim i włoskim w serwisie Amazon na całym świecie. Poznaj wydanie Plus i kup je w serwisie Amazon.

# Wesprzyj projekt

Jeśli bezpłatna książka pomogła Ci naprawić błąd, zrozumieć trudne zagadnienie lub rozwinąć karierę, rozważ wsparcie mojej pracy poprzez wpłatę dowolnej kwoty — sugerowane wsparcie wynosi **5 USD** — albo postawienie mi kawy.

Twoje wsparcie pomaga mi aktualizować treść i rozszerzać ją o nowe przykłady, jaśniejsze objaśnienia oraz dodatkowe praktyczne wskazówki.







# Tłumaczenia

Ta książka została przetłumaczona na kilka języków, w tym:

angielski

bułgarski

niemiecki

francuski

indonezyjski

włoski

japoński

koreański

polski

portugalski (Brazylia)

szwedzki

turecki

Vietnamese

chiński

hiszpański

tajski

rosyjski

arabski Czeski

# Pliki do pobrania i witryna

Możesz także pobrać wersję EPUB:

https://github.com/gibbok/typescript-book/tree/main/downloads

Wersja online jest dostępna pod adresem:

https://gibbok.github.io/typescript-book

# Spis treści


- The Concise TypeScript Book
  - Wesprzyj projekt
  - Tłumaczenia
  - Pliki do pobrania i witryna
  - Spis treści
  - Wprowadzenie
  - O autorze
  - Wprowadzenie do TypeScript
    - Czym jest TypeScript?
    - Dlaczego TypeScript?
    - TypeScript i JavaScript
    - Generowanie kodu przez TypeScript
    - Nowoczesny JavaScript już dziś (transpilacja do starszych wersji)
  - Pierwsze kroki z TypeScript
    - Instalacja
    - Konfiguracja
    - Plik konfiguracyjny TypeScript
      - target
      - lib

        - strict
        - module
        - moduleResolution
        - esModuleInterop
        - jsx
        - skipLibCheck
        - files
        - include
        - exclude
    - importHelpers
    - Wskazówki dotyczące migracji do TypeScript
- Poznawanie systemu typów
    - Usługa językowa TypeScript
    - Typowanie strukturalne
    - Podstawowe reguły porównywania w TypeScript
    - Typy jako zbiory
    - Przypisywanie typu: deklaracje typów i asercje typów
        - Deklaracja typu
        - Asercja typu
        - Deklaracje otoczenia
    - Kontrola właściwości i kontrola nadmiarowych właściwości
    - Typy słabe
    - Ścisła kontrola literałów obiektowych (świeżość)
    - Wnioskowanie typów
    - Bardziej zaawansowane wnioskowanie
    - Poszerzanie typów
    - Const
        - Modyfikator Const parametrów typu
        - Asercja Const
    - Jawna adnotacja typu

        - Zawężanie typów
            - Warunki
            - Zgłoszenie wyjątku lub zwrócenie wartości
            - Unia dyskryminowana
            - Typy strażnicze zdefiniowane przez użytkownika
            - Zawężanie za pomocą switch-true
    - Typy pierwotne
        - string
        - boolean
        - number
        - bigint
        - Symbol
        - null i undefined
        - Tablica
        - any
    - Adnotacje typów
    - Właściwości opcjonalne
    - Właściwości tylko do odczytu
    - Sygnatury indeksowe
    - Rozszerzanie typów
    - Typy literałowe
    - Wnioskowanie typów literałowych
    - strictNullChecks
    - Wyliczenia
        - Wyliczenia liczbowe
        - Wyliczenia ciągów znaków
        - Wyliczenia stałe
        - Mapowanie odwrotne
        - Wyliczenia ambientowe
        - Elementy wyliczane i stałe
    - Zawężanie typów

        - Strażniki typów typeof
        - Zawężanie na podstawie prawdziwości
        - Zawężanie na podstawie równości
        - Zawężanie za pomocą operatora in
        - Zawężanie za pomocą instanceof
    - Przypisania
    - Analiza przepływu sterowania
    - Predykaty typów
    - Unie dyskryminowane
    - Specjalny typ never
    - Sprawdzanie kompletności
    - Typy obiektowe
    - Typ krotki (anonimowy)
    - Nazwany typ krotki (z etykietami)
    - Krotka o stałej długości
    - Typ unii
    - Typy przecięcia
    - Indeksowanie typów
    - Typ na podstawie wartości
    - Typ na podstawie wartości zwracanej przez funkcję
    - Typ na podstawie modułu
    - Typy mapowane
    - Modyfikatory typów mapowanych
    - Typy warunkowe
    - Dystrybucyjne typy warunkowe
    - Wnioskowanie typu infer w typach warunkowych
    - Predefiniowane typy warunkowe
    - Szablonowe typy unii
    - Typ any
    - Typ unknown
    - Typ void
    - Typ never

- Interfejs i typ
  - Podstawowa składnia
  - Typy podstawowe
  - Obiekty i interfejsy
  - Typy sumy i przecięcia
- Wbudowane typy proste
- Typowe wbudowane obiekty JavaScript
- Przeciążenia
- Scalanie i rozszerzanie
- Różnice między typem a interfejsem
- Klasa
  - Podstawowa składnia klasy
  - Konstruktor
  - Konstruktory prywatne i chronione
  - Modyfikatory dostępu
  - Gettery i settery
  - Automatyczne akcesory w klasach
  - this
  - Właściwości parametrów
  - Klasy abstrakcyjne
  - Z typami generycznymi
  - Dekoratory
    - Dekoratory klas
    - Dekorator właściwości
    - Dekorator metody
    - Dekoratory getterów i setterów
    - Metadane dekoratorów
  - Dziedziczenie
  - Elementy statyczne
  - Inicjalizacja właściwości
  - Przeciążanie metod
- Typy generyczne

        - Typ generyczny
        - Klasy generyczne
        - Ograniczenia typów generycznych
        - Kontekstowe zawężanie typów generycznych
    - Wymazywane typy strukturalne
    - Przestrzenie nazw
    - Symbole
    - Dyrektywy z potrójnym ukośnikiem
    - Manipulowanie typami
        - Tworzenie typów na podstawie innych typów
        - Typy dostępu indeksowanego
        - Typy narzędziowe
            - Awaited<T>
            - Partial<T>
            - Required<T>
            - Readonly<T>
            - Record<K, T>
            - Pick<T, K>
            - Omit<T, K>
            - Exclude<T, U>
            - Extract<T, U>
            - NonNullable<T>
            - Parameters<T>
            - ConstructorParameters<T>
            - ReturnType<T>
            - InstanceType<T>
            - ThisParameterType<T>
            - OmitThisParameter<T>
            - ThisType<T>
            - Uppercase<T>
            - Lowercase<T>
            - Capitalize<T>

        - Uncapitalize<T>
        - NoInfer<T>
- Inne
    - Obsługa błędów i wyjątków
    - Klasy domieszkowe
    - Asynchroniczne funkcje języka
    - Iteratory i generatory
    - Dokumentacja TsDocs JSDoc
    - @types
    - JSX
    - Moduły ES6
    - Operator potęgowania ES7
    - Instrukcja for-await-of
    - Metawłaściwość new.target
    - Wyrażenia importu dynamicznego
    - “tsc –watch”
    - Operator asercji non-null
    - Deklaracje z wartościami domyślnymi
    - Łańcuch opcjonalny
    - Operator koalescencji null
    - Typy literałów szablonowych
    - Przeciążanie funkcji
    - Typy rekurencyjne
    - Rekurencyjne typy warunkowe
    - Obsługa modułów ECMAScript w Node
    - Funkcje asercji
    - Wariadyczne typy krotek
    - Typy opakowujące
    - Kowariancja i kontrawariancja w TypeScript
        - Opcjonalne adnotacje wariancji parametrów typów

- Sygnatury indeksowe ze wzorcami ciągów szablonowych
- Operator satisfies
- Importy i eksporty wyłącznie typów
- Deklaracja using i jawne zarządzanie zasobami
  - Deklaracja await using
- Atrybuty importu
- Sprawdzanie składni wyrażeń regularnych
- import defer


# Wprowadzenie

Witamy w The Concise TypeScript Book! Ten przewodnik zapewni Ci podstawową wiedzę i praktyczne umiejętności niezbędne do efektywnego programowania w TypeScript. Poznasz najważniejsze pojęcia i techniki tworzenia czystego, niezawodnego kodu. Niezależnie od tego, czy jesteś osobą początkującą, czy doświadczonym programistą, ta książka stanowi zarówno kompleksowy przewodnik, jak i poręczne źródło informacji, które pomoże Ci wykorzystać możliwości TypeScript w swoich projektach.

Ta książka omawia TypeScript 7.0.

# O autorze

Simone Poggiali to doświadczony Staff Engineer, który z pasją tworzy profesjonalny kod od lat 90. W trakcie swojej międzynarodowej kariery uczestniczył w wielu projektach dla szerokiego grona klientów — od startupów po duże organizacje. Z jego wiedzy i zaangażowania skorzystały znane firmy, takie jak HelloFresh, Siemens, O2, Leroy Merlin i Snowplow.

Z Simone Poggialim można skontaktować się na następujących platformach:

- LinkedIn: https://www.linkedin.com/in/simone-poggiali
- GitHub: https://github.com/gibbok
- Bluesky: https://bsky.app/profile/gibbok.bsky.social
- E-mail: gibbok.coding  gmail.com

Pełna lista współtwórców: https://github.com/gibbok/typescript-book/graphs/contributors

# Wprowadzenie do TypeScript

## Czym jest TypeScript?

TypeScript jest językiem programowania z silnym typowaniem, zbudowanym na bazie języka JavaScript. Został pierwotnie zaprojektowany przez Andersa Hejlsberga w 2012 roku, a obecnie jest rozwijany i utrzymywany przez firmę Microsoft jako projekt open source.

TypeScript jest kompilowany do JavaScript i może być wykonywany w dowolnym środowisku uruchomieniowym JavaScript (np. w przeglądarce lub w środowisku Node.js na serwerze).

Obsługuje wiele paradygmatów programowania, takich jak programowanie funkcyjne, generyczne, imperatywne i obiektowe oraz jest językiem kompilowanym (transpilowanym), który przed wykonaniem jest przekształcany w JavaScript.

## Dlaczego TypeScript?

TypeScript jest językiem z silnym typowaniem, który pomaga zapobiegać częstym błędom programistycznym i unikać określonych rodzajów błędów czasu wykonywania przed uruchomieniem programu.

Język z silnym typowaniem pozwala programiście określać różne ograniczenia i zachowania programu w definicjach typów danych, ułatwiając weryfikowanie poprawności oprogramowania i zapobieganie defektom. Jest to szczególnie cenne w aplikacjach o dużej skali.

Niektóre zalety TypeScript:

- Typowanie statyczne, opcjonalnie silne
- Wnioskowanie typów
- Dostęp do funkcji ES6 i ES7
- Zgodność międzyplatformowa i zgodność z różnymi przeglądarkami
- Obsługa narzędziowa z IntelliSense

## TypeScript i JavaScript

Kod TypeScript zapisuje się w plikach `.ts` lub `.tsx`, natomiast kod JavaScript — w plikach `.js` lub `.jsx`.

Pliki z rozszerzeniem `.tsx` lub `.jsx` mogą zawierać rozszerzenie składni JavaScript JSX, używane w React do tworzenia interfejsów użytkownika.

Pod względem składni TypeScript jest typowanym nadzbiorem JavaScript (ECMAScript 2015). Cały kod JavaScript jest poprawnym kodem TypeScript, ale odwrotna zależność nie zawsze zachodzi.

Rozważmy na przykład funkcję w pliku JavaScript z rozszerzeniem `.js`, taką jak poniższa:

```
const sum = (a, b) => a + b;
```

Funkcję można przekonwertować i użyć w TypeScript, zmieniając rozszerzenie pliku na `.ts`. Jeśli jednak ta sama funkcja zostanie opatrzona adnotacjami typów TypeScript, nie można jej wykonać w żadnym środowisku uruchomieniowym JavaScript bez wcześniejszej kompilacji. Poniższy kod TypeScript spowoduje błąd składni, jeśli nie zostanie skompilowany:

```
const sum = (a: number, b: number): number => a + b;
```

TypeScript zaprojektowano tak, aby wykrywał potencjalne błędy czasu wykonywania w czasie kompilacji, umożliwiając programistom wyrażanie intencji za pomocą adnotacji typów. Ponadto dzięki wnioskowaniu typów TypeScript potrafi wykrywać niektóre problemy nawet wtedy, gdy nie podano jawnych adnotacji typów. Na przykład poniższy fragment kodu nie określa żadnych typów TypeScript:

```
const items = [{ x: 1 }, { x: 2 }];
const result = items.filter(item => item.y);
```

W tym przypadku TypeScript wykrywa błąd i zgłasza:

```
Property 'y' does not exist on type '{ x: number; }'.
```

Na system typów TypeScript duży wpływ ma zachowanie JavaScript w czasie wykonywania. Na przykład operator dodawania (+), który w JavaScript może wykonywać konkatenację ciągów znaków albo dodawanie liczb, jest modelowany w TypeScript w ten sam sposób:

```
const result = '1' + 1; // Result is of type string
```

Zespół TypeScript świadomie zdecydował się oznaczać nietypowe użycie JavaScript jako błędy. Rozważmy na przykład następujący poprawny kod JavaScript:

```
const result = 1 + true; // In JavaScript, the result is equal to 2
```

TypeScript zgłasza jednak błąd:

```
Operator '+' cannot be applied to types 'number' and 'boolean'.
```

Ten błąd występuje, ponieważ TypeScript ściśle egzekwuje zgodność typów i w tym przypadku rozpoznaje nieprawidłową operację między liczbą a wartością logiczną.

## Generowanie kodu przez TypeScript

Kompilator TypeScript ma dwa główne zadania: sprawdzanie błędów typów i kompilowanie do JavaScript. Te dwa procesy są od siebie niezależne. Typy nie wpływają na wykonywanie kodu w środowisku uruchomieniowym JavaScript, ponieważ są całkowicie usuwane podczas kompilacji. TypeScript może wygenerować kod JavaScript nawet w przypadku wystąpienia błędów typów. Oto przykład kodu TypeScript z błędem typu:

```
const add = (a: number, b: number): number => a + b;
const result = add('x', 'y'); // Argument of type 'string' is not assignable to parameter of type 'number'.
```

Mimo to może wygenerować wykonywalny kod JavaScript:

```
'use strict';
const add = (a, b) => a + b;
const result = add('x', 'y'); // xy
```

Nie można sprawdzać typów TypeScript w czasie wykonywania. Na przykład:

```
interface Animal {
    name: string;
}
interface Dog extends Animal {
    bark: () => void;
}
interface Cat extends Animal {
    meow: () => void;
}
const makeNoise = (animal: Animal) => {
    if (animal instanceof Dog) {
        // 'Dog' only refers to a type, but is being used as a
value here.
        // ...
    }
};
```

Ponieważ typy są usuwane po kompilacji, nie ma możliwości wykonania tego kodu w JavaScript. Aby rozpoznawać typy w czasie wykonywania, należy użyć innego mechanizmu. TypeScript udostępnia kilka możliwości, z których często stosowaną jest „unia znakowana”. Na przykład:

```
interface Dog {
    kind: 'dog'; // Tagged union
    bark: () => void;
}
interface Cat {
    kind: 'cat'; // Tagged union
    meow: () => void;
}
type Animal = Dog | Cat;

const makeNoise = (animal: Animal) => {
```

```
    if (animal.kind === 'dog') {
        animal.bark();
    } else {
        animal.meow();
    }
};

const dog: Dog = {
    kind: 'dog',
    bark: () => console.log('bark'),
};
makeNoise(dog);
```

Właściwość „kind” jest wartością, której można użyć w czasie wykonywania do rozróżniania obiektów w JavaScript.

Możliwe jest również, że wartość w czasie wykonywania ma typ inny niż zadeklarowany w deklaracji typu. Może się tak zdarzyć na przykład wtedy, gdy programista błędnie zinterpretował typ interfejsu API i opatrzył go nieprawidłową adnotacją.

TypeScript jest nadzbiorem JavaScript, dlatego słowa kluczowego „class” można używać jako typu oraz wartości w czasie wykonywania.

```
class Animal {
    constructor(public name: string) {}
}
class Dog extends Animal {
    constructor(
        public name: string,
        public bark: () => void
    ) {
        super(name);
    }
}
class Cat extends Animal {
```

```
    constructor(
        public name: string,
        public meow: () => void
    ) {
        super(name);
    }
}
type Mammal = Dog | Cat;

const makeNoise = (mammal: Mammal) => {
    if (mammal instanceof Dog) {
        mammal.bark();
    } else {
        mammal.meow();
    }
};

const dog = new Dog('Fido', () => console.log('bark'));
makeNoise(dog);
```

W JavaScript „class” ma właściwość „prototype”, a operatora „instanceof” można użyć do sprawdzenia, czy właściwość prototype konstruktora występuje w dowolnym miejscu łańcucha prototypów obiektu.

TypeScript nie wpływa na wydajność w czasie wykonywania, ponieważ wszystkie typy zostają usunięte. Wprowadza jednak pewien narzut podczas kompilacji.

## Nowoczesny JavaScript już dziś (transpilacja do starszych wersji)

TypeScript może kompilować kod do dowolnej wydanej wersji JavaScript, począwszy od ECMAScript 3 (1999). Oznacza to, że TypeScript może transpilować kod korzystający z najnowszych funkcji JavaScript do starszych wersji — proces ten nazywa się

transpilacją do starszych wersji (downleveling). Pozwala to korzystać z nowoczesnego JavaScript przy zachowaniu maksymalnej zgodności ze starszymi środowiskami uruchomieniowymi.

Należy pamiętać, że podczas transpilacji do starszej wersji JavaScript TypeScript może wygenerować kod, który powoduje narzut wydajnościowy w porównaniu z natywnymi implementacjami.

Oto niektóre z nowoczesnych funkcji JavaScript, których można używać w TypeScript:

- Moduły ECMAScript zamiast wywołań zwrotnych „define” w stylu AMD lub instrukcji „require” CommonJS.
- Klasy zamiast prototypów.
- Deklarowanie zmiennych za pomocą „let” lub „const” zamiast „var”.
- Pętla „for-of” lub metoda „.forEach” zamiast tradycyjnej pętli „for”.
- Funkcje strzałkowe zamiast wyrażeń funkcyjnych.
- Przypisanie destrukturyzujące.
- Skrócone nazwy właściwości i metod oraz obliczane nazwy właściwości.
- Domyślne parametry funkcji.

Dzięki wykorzystaniu tych nowoczesnych funkcji JavaScript programiści mogą pisać w TypeScript bardziej ekspresyjny i zwięzły kod.

# Pierwsze kroki z TypeScript

## Instalacja

Visual Studio Code oferuje doskonałą obsługę języka TypeScript, ale nie zawiera kompilatora TypeScript. Aby zainstalować kompilator TypeScript, można użyć menedżera pakietów, takiego jak npm lub yarn:

```
npm install typescript --save-dev
```

lub

```
yarn add typescript --dev
```

Należy pamiętać o dodaniu wygenerowanego pliku blokady do repozytorium, aby każdy członek zespołu korzystał z tej samej wersji TypeScript.

Aby uruchomić kompilator TypeScript, można użyć następujących poleceń:

```
npx tsc
```

lub

```
yarn tsc
```

Zaleca się instalowanie TypeScript na poziomie projektu, a nie globalnie, ponieważ zapewnia to bardziej przewidywalny proces kompilacji. Jednak do jednorazowego użycia można zastosować następujące polecenie:

```
npx tsc
```

lub zainstalować go globalnie:

```
npm install -g typescript
```

Jeśli używasz Microsoft Visual Studio, możesz uzyskać TypeScript jako pakiet NuGet dla swoich projektów MSBuild. W konsoli menedżera pakietów NuGet uruchom następujące polecenie:

```
Install-Package Microsoft.TypeScript.MSBuild
```

Podczas instalacji TypeScript instalowane są dwa pliki wykonywalne: „tsc” jako kompilator TypeScript oraz „tsserver” jako samodzielny serwer TypeScript. Samodzielny serwer zawiera kompilator i usługi językowe, które mogą być wykorzystywane przez edytory oraz zintegrowane środowiska programistyczne do zapewniania inteligentnego uzupełniania kodu.

Dostępnych jest również kilka transpilerów zgodnych z TypeScript, takich jak Babel (za pośrednictwem wtyczki) lub swc. Można ich używać do konwertowania kodu TypeScript na inne języki docelowe lub wersje.

TypeScript 7.0 został przepisany w Go jako natywna implementacja kompilatora i usługi językowej. Wykorzystuje wielowątkowość z pamięcią współdzieloną oraz inne optymalizacje, aby przyspieszyć pełne kompilacje i funkcje edytora, skracając czas oczekiwania na informacje zwrotne podczas programowania.

Niektóre funkcje wydajnościowe TypeScript 7.0 można dostosować. Sprawdzanie typów może działać w równoległych procesach roboczych za pomocą opcji `--checkers`; większa liczba procesów może przyspieszyć duże projekty, ale zużywa więcej pamięci. Przebudowany tryb `--watch` usprawnia monitorowanie zmian w plikach na różnych platformach. TypeScript 7.0 nie

zawiera jeszcze interfejsu API kompilatora (stan na lipiec 2026), dlatego narzędzia, które nadal wymagają interfejsu API TypeScript 6.0, mogą działać równolegle z TypeScript 7.0 dzięki użyciu `@typescript/typescript6` lub aliasów npm.

## Konfiguracja

TypeScript można skonfigurować za pomocą opcji interfejsu wiersza poleceń tsc lub specjalnego pliku konfiguracyjnego o nazwie tsconfig.json, umieszczonego w katalogu głównym projektu.

Aby wygenerować plik tsconfig.json ze wstępnie zdefiniowanymi zalecanymi ustawieniami, można użyć następującego polecenia:

```
tsc --init
```

Podczas lokalnego wykonywania polecenia `tsc` TypeScript skompiluje kod przy użyciu konfiguracji określonej w najbliższym pliku tsconfig.json.

Oto kilka przykładów poleceń interfejsu wiersza poleceń uruchamianych z ustawieniami domyślnymi:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

## Plik konfiguracyjny TypeScript

Plik tsconfig.json służy do konfigurowania kompilatora TypeScript (tsc). Zwykle umieszcza się go w katalogu głównym projektu razem z plikiem `package.json`.

Uwagi:

- Plik tsconfig.json akceptuje komentarze, mimo że ma format json.
- Zaleca się używanie tego pliku konfiguracyjnego zamiast opcji wiersza poleceń.

Pod poniższym łączem można znaleźć pełną dokumentację i jej schemat:

https://www.typescriptlang.org/tsconfig

https://www.typescriptlang.org/tsconfig/

Poniżej przedstawiono listę często używanych i przydatnych opcji konfiguracji:

**target**

Właściwość „target” służy do określania wersji ECMAScript, do której kod TypeScript ma zostać wyemitowany lub skompilowany. W przypadku nowoczesnych przeglądarek dobrym wyborem jest ES6. Uwaga: obsługa ES5 została uznana za przestarzałą w TypeScript 6.0 i nie jest już dostępna w TypeScript 7.0.

**lib**

Właściwość „lib” służy do określania plików bibliotek, które mają zostać uwzględnione podczas kompilacji. TypeScript automatycznie uwzględnia interfejsy API dla funkcjonalności określonych we właściwości „target”, ale można pominąć lub wybrać konkretne biblioteki zależnie od potrzeb. Na przykład podczas pracy nad projektem serwerowym można wykluczyć bibliotekę „DOM”, która jest przydatna tylko w środowisku przeglądarki.

**strict**

Opcja „strict” zwiększa bezpieczeństwo typów przez włączenie bardziej rygorystycznych kontroli. Począwszy od TypeScript 6.0 jest domyślnie włączona; w przeciwnym razie należy jawnie ustawić ją na true w pliku tsconfig.json. Włączenie opcji „strict” sprawia, że TypeScript:

- Emituje kod z użyciem „use strict” dla każdego pliku źródłowego.
- Uwzględnia „null” i „undefined” w procesie sprawdzania typów.
- Wyłącza użycie typu „any”, gdy nie ma adnotacji typów.
- Zgłasza błąd w przypadku użycia wyrażenia „this”, które w przeciwnym razie implikowałoby typ „any”.

**module**

Właściwość „module” ustawia system modułów obsługiwany przez skompilowany program. W czasie wykonywania mechanizm ładujący moduły służy do lokalizowania i wykonywania zależności na podstawie określonego systemu modułów.

Najczęściej używanymi mechanizmami ładowania modułów w JavaScript są CommonJS w środowisku Node.js dla aplikacji serwerowych oraz RequireJS dla modułów AMD w aplikacjach internetowych działających w przeglądarce. TypeScript może emitować kod dla różnych systemów modułów, w tym UMD, System, ESNext, ES2015/ES6 i ES2020. System modułów należy wybrać na podstawie środowiska docelowego i dostępnego w nim mechanizmu ładowania modułów.

Uwaga: obsługa starszych systemów modułów (AMD, UMD, SystemJS) została uznana za przestarzałą w TypeScript 6.0 i nie jest już dostępna w TypeScript 7.0.

**moduleResolution**

Właściwość „moduleResolution” określa strategię rozwiązywania modułów. W nowoczesnym kodzie TypeScript należy używać „nodenext” lub „bundler”. Strategia „classic” jest używana tylko w starych wersjach TypeScript (sprzed wersji 1.6).

**esModuleInterop**

Właściwość „esModuleInterop” umożliwia domyślne importy z modułów CommonJS, które nie eksportowały za pomocą właściwości „default”; ta właściwość zapewnia kod zgodności w wygenerowanym kodzie JavaScript. Po włączeniu tej opcji można używać `import MyLibrary from "my-library"` zamiast `import * as MyLibrary from "my-library"`.

Opcja „esModuleInterop” była pierwotnie opcjonalna, aby uniknąć zmian niezgodnych wstecznie, ale od dawna jest

zalecanym ustawieniem domyślnym. Jej wyłączenie może powodować subtelne problemy w czasie wykonywania podczas używania CommonJS z ESM. Uwaga: począwszy od TypeScript 6.0, to bezpieczniejsze zachowanie interoperacyjności jest zawsze włączone.

W TypeScript 6.0 niektóre starsze opcje konfiguracji i formy składni zostały uznane za przestarzałe lub w okresie przejściowym zachowywały dotychczasowe działanie. W TypeScript 7.0 powodują one twarde błędy lub nie wywołują żadnego działania.

Elementy uznane za przestarzałe, które obecnie powodują twarde błędy i nie wywołują żadnego działania, to:

- `target: es5`
- `downlevelIteration`
- `moduleResolution: node/node10`
- `module: amd/umd/systemjs/none`
- `baseUrl`
- `moduleResolution: classic`
- wyłączenie `esModuleInterop` lub `allowSyntheticDefaultImports`
- wyłączenie `alwaysStrict`
- słowo kluczowe `module` w deklaracjach przestrzeni nazw
- `asserts` w importach
- `/// <reference no-default-lib />` przy włączonej opcji `skipDefaultLibCheck`
- ścieżki plików przekazane w interfejsie wiersza poleceń przy lokalnym pliku `tsconfig.json`, chyba że użyto opcji `--ignoreConfig`

**jsx**

Właściwość „jsx” ma zastosowanie wyłącznie do plików .tsx używanych w ReactJS i steruje sposobem kompilowania konstrukcji JSX do JavaScript. Często używaną opcją jest „preserve”, która kompiluje kod do pliku .jsx, pozostawiając JSX bez zmian, aby można go było przekazać do innych narzędzi, takich jak Babel, w celu dalszych transformacji.

**skipLibCheck**

Właściwość „skipLibCheck” zapobiega sprawdzaniu przez TypeScript typów całych importowanych pakietów innych firm. Skraca to czas kompilacji projektu. TypeScript nadal sprawdza kod względem definicji typów udostępnionych przez te pakiety.

**files**

Właściwość „files” wskazuje kompilatorowi listę plików, które muszą zawsze być uwzględnione w programie.

**include**

Właściwość „include” wskazuje kompilatorowi listę plików, które mają zostać uwzględnione. Właściwość ta umożliwia stosowanie wzorców podobnych do globów, takich jak „*” *dla dowolnego podkatalogu*, „” dla dowolnej nazwy pliku oraz „?” dla znaków opcjonalnych.

**exclude**

Właściwość „exclude” wskazuje kompilatorowi listę plików, które nie powinny być uwzględniane w kompilacji. Może ona

obejmować pliki takie jak „node_modules” lub pliki testowe. Uwaga: plik tsconfig.json dopuszcza komentarze.

## importHelpers

TypeScript używa kodu funkcji pomocniczych podczas generowania kodu dla niektórych zaawansowanych funkcji JavaScript lub funkcji transpilowanych do starszych wersji. Domyślnie te funkcje pomocnicze są powielane w plikach, które z nich korzystają. Opcja `importHelpers` importuje je zamiast tego z modułu `tslib`, dzięki czemu wygenerowany kod JavaScript jest bardziej wydajny.

## Wskazówki dotyczące migracji do TypeScript

W przypadku dużych projektów zaleca się stopniowe przejście, podczas którego kod TypeScript i JavaScript będą początkowo współistnieć. Tylko małe projekty można zmigrować do TypeScript za jednym razem.

Pierwszym krokiem tego przejścia jest wprowadzenie TypeScript do procesu kompilacji. Można to zrobić za pomocą opcji kompilatora „allowJs”, która pozwala plikom .ts i .tsx współistnieć z istniejącymi plikami JavaScript. Ponieważ TypeScript używa typu „any” dla zmiennej, gdy nie może wywnioskować jej typu z plików JavaScript, na początku migracji zaleca się wyłączenie opcji „noImplicitAny” w opcjach kompilatora.

Drugim krokiem jest upewnienie się, że testy JavaScript działają razem z plikami TypeScript, aby można było uruchamiać testy podczas konwertowania każdego modułu. Jeśli używasz Jest,

rozważ użycie `ts-jest`, które umożliwia testowanie projektów TypeScript za pomocą Jest.

Trzecim krokiem jest dodanie do projektu deklaracji typów dla bibliotek innych firm. Deklaracje te można znaleźć w pakiecie z biblioteką lub w repozytorium DefinitelyTyped. Można je wyszukać pod adresem https://www.typescriptlang.org/dt/search i zainstalować za pomocą polecenia:

```
npm install --save-dev @types/package-name
```

lub

```
yarn add --dev @types/package-name
```

Czwartym krokiem jest migrowanie moduł po module zgodnie z podejściem oddolnym, zaczynając od liści grafu zależności. Chodzi o to, aby zacząć konwertowanie modułów, które nie zależą od innych modułów. Do wizualizacji grafów zależności można użyć narzędzia „madge”.

Dobrymi kandydatami do początkowej konwersji są moduły funkcji narzędziowych oraz kod związany z zewnętrznymi interfejsami API lub specyfikacjami. Możliwe jest automatyczne generowanie definicji typów TypeScript na podstawie kontraktów Swagger oraz schematów GraphQL lub JSON, które można następnie uwzględnić w projekcie.

Gdy nie są dostępne żadne specyfikacje ani oficjalne schematy, można wygenerować typy na podstawie nieprzetworzonych danych, takich jak dane JSON zwracane przez serwer. Zaleca się jednak generowanie typów na podstawie specyfikacji, a nie danych, aby uniknąć pominięcia przypadków brzegowych.

Podczas migracji należy powstrzymać się od refaktoryzacji kodu i skupić wyłącznie na dodawaniu typów do modułów.

Piątym krokiem jest włączenie opcji „noImplicitAny”, która wymusi, aby wszystkie typy były znane i zdefiniowane, zapewniając lepsze środowisko pracy z TypeScript w projekcie.

Podczas migracji można używać dyrektywy `@ts-check`, która włącza sprawdzanie typów TypeScript w pliku JavaScript. Dyrektywa ta zapewnia luźniejszą wersję sprawdzania typów i może być początkowo używana do identyfikowania problemów w plikach JavaScript. Gdy dyrektywa `@ts-check` jest umieszczona w pliku, TypeScript próbuje wywnioskować definicje za pomocą komentarzy w stylu JSDoc. Należy jednak rozważyć używanie adnotacji JSDoc wyłącznie na bardzo wczesnym etapie migracji.

Rozważ pozostawienie domyślnej wartości opcji `noEmitOnError` w pliku tsconfig.json jako false. Pozwoli to generować kod źródłowy JavaScript nawet w przypadku zgłoszenia błędów.

# Poznawanie systemu typów

## Usługa językowa TypeScript

Usługa językowa TypeScript, znana również jako tsserver, oferuje różne funkcje, takie jak raportowanie błędów, diagnostyka, kompilacja przy zapisie, zmiana nazw, przechodzenie do definicji, listy uzupełnień, podpowiedzi dotyczące sygnatur i inne. Jest używana przede wszystkim przez zintegrowane środowiska programistyczne (IDE) do obsługi IntelliSense. Płynnie integruje się z Visual Studio Code i

jest wykorzystywana przez narzędzia takie jak Conquer of Completion (Coc).

Programiści mogą korzystać ze specjalnego API i tworzyć własne wtyczki usługi językowej, aby usprawnić pracę z TypeScript podczas edycji kodu. Może to być szczególnie przydatne do implementowania specjalnych funkcji lintowania lub włączania automatycznego uzupełniania dla niestandardowego języka szablonów.

Przykładem niestandardowej wtyczki używanej w rzeczywistych projektach jest „typescript-styled-plugin", która zapewnia raportowanie błędów składniowych oraz obsługę IntelliSense dla właściwości CSS w komponentach stylizowanych.

Więcej informacji i przewodniki szybkiego startu można znaleźć w oficjalnej witrynie Wiki TypeScript w serwisie GitHub: https://github.com/microsoft/TypeScript/wiki/.

## Typowanie strukturalne

TypeScript opiera się na strukturalnym systemie typów. Oznacza to, że zgodność i równoważność typów są określane na podstawie rzeczywistej struktury lub definicji typu, a nie jego nazwy czy miejsca deklaracji, jak w nominalnych systemach typów, takich jak C# lub C.

Strukturalny system typów TypeScript został zaprojektowany na podstawie sposobu, w jaki działa dynamiczne kacze typowanie w JavaScript w czasie wykonywania.

Poniższy przykład jest poprawnym kodem TypeScript. Jak widać, „X” i „Y” mają ten sam element składowy „a”, mimo że ich deklaracje mają różne nazwy. Typy są określane na podstawie ich struktur, a ponieważ w tym przypadku struktury są takie same, typy są zgodne i poprawne.

```
type X = {
    a: string;
};
type Y = {
    a: string;
};
const x: X = { a: 'a' };
const y: Y = x; // Valid
```

## Podstawowe reguły porównywania w TypeScript

Proces porównywania w TypeScript jest rekurencyjny i wykonywany dla typów zagnieżdżonych na dowolnym poziomie.

Typ „X” jest zgodny z typem „Y”, jeśli „Y” ma co najmniej te same elementy składowe co „X”.

```
type X = {
    a: string;
};
const y = { a: 'A', b: 'B' }; // Valid, as it has at least the
same members as X
const r: X = y;
```

Parametry funkcji są porównywane według typów, a nie według ich nazw:

```
type X = (a: number) => void;
type Y = (a: number) => void;
```

```
let x: X = (j: number) => undefined;
let y: Y = (k: number) => undefined;
y = x; // Valid
x = y; // Valid
```

Typy zwracane przez funkcje muszą być takie same:

```
type X = (a: number) => undefined;
type Y = (a: number) => number;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => 1;
y = x; // Invalid
x = y; // Invalid
```

Typ zwracany przez funkcję źródłową musi być podtypem typu zwracanego przez funkcję docelową:

```
let x = () => ({ a: 'A' });
let y = () => ({ a: 'A', b: 'B' });
x = y; // Valid
y = x; // Invalid member b is missing
```

Pomijanie parametrów funkcji jest dozwolone, ponieważ jest to powszechna praktyka w JavaScript, na przykład podczas używania „Array.prototype.map()”:

```
[1, 2, 3].map((element, _index, _array) => element + 'x');
```

Dlatego poniższe deklaracje typów są całkowicie poprawne:

```
type X = (a: number) => undefined;
type Y = (a: number, b: number) => undefined;
let x: X = (a: number) => undefined;
let y: Y = (a: number) => undefined; // Missing b parameter
y = x; // Valid
```

Wszelkie dodatkowe parametry opcjonalne typu źródłowego są poprawne:

```
type X = (a: number, b?: number, c?: number) => undefined;
type Y = (a: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; //Valid
```

Wszelkie parametry opcjonalne typu docelowego, którym nie odpowiadają parametry w typie źródłowym, są poprawne i nie powodują błędu:

```
type X = (a: number) => undefined;
type Y = (a: number, b?: number) => undefined;
let x: X = a => undefined;
let y: Y = a => undefined;
y = x; // Valid
x = y; // Valid
```

Parametr resztowy jest traktowany jako nieskończona seria parametrów opcjonalnych:

```
type X = (a: number, ...rest: number[]) => undefined;
let x: X = a => undefined; //valid
```

Funkcje z przeciążeniami są poprawne, jeśli sygnatura przeciążenia jest zgodna z sygnaturą implementacji:

```
function x(a: string): void;
function x(a: string, b: number): void;
function x(a: string, b?: number): void {
    console.log(a, b);
}
x('a'); // Valid
x('a', 1); // Valid

function y(a: string): void; // Invalid, not compatible with implementation signature
function y(a: string, b: number): void;
```

```
function y(a: string, b: number): void {
    console.log(a, b);
}
y('a');
y('a', 1);
```

Porównanie parametrów funkcji kończy się powodzeniem, jeśli parametry źródłowe i docelowe można przypisać do nadtypów lub podtypów (biwariancja).

```
// Supertype
class X {
    a: string;
    constructor(value: string) {
        this.a = value;
    }
}
// Subtype
class Y extends X {}
// Subtype
class Z extends X {}

type GetA = (x: X) => string;
const getA: GetA = x => x.a;

// Bivariance does accept supertypes
console.log(getA(new X('x'))); // Valid
console.log(getA(new Y('Y'))); // Valid
console.log(getA(new Z('z'))); // Valid
```

Typy wyliczeniowe są porównywalne i zgodne z liczbami, a liczby z typami wyliczeniowymi, ale porównywanie wartości pochodzących z różnych typów wyliczeniowych jest niepoprawne.

```
enum X {
    A,
```

```
    B,
}
enum Y {
    A,
    B,
    C,
}
const xa: number = X.A; // Valid
const ya: Y = 0; // Valid
X.A === Y.A; // Invalid
```

Instancje klasy podlegają kontroli zgodności swoich prywatnych i chronionych elementów składowych:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}

class Y {
    private a: string;
    constructor(value: string) {
        this.a = value;
    }
}

let x: X = new Y('y'); // Invalid
```

Kontrola porównawcza nie uwzględnia różnic w hierarchii dziedziczenia, na przykład:

```
class X {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
```

```
}
class Y extends X {
    public a: string;
    constructor(value: string) {
        super(value);
        this.a = value;
    }
}
class Z {
    public a: string;
    constructor(value: string) {
        this.a = value;
    }
}
let x: X = new X('x');
let y: Y = new Y('y');
let z: Z = new Z('z');
x === y; // Valid
x === z; // Valid even if z is from a different inheritance hierarchy
```

Typy generyczne są porównywane na podstawie swoich struktur, z uwzględnieniem typu wynikowego otrzymanego po zastosowaniu parametru typu. Porównywany jest wyłącznie końcowy wynik jako typ niegeneryczny.

```
interface X<T> {
    a: T;
}
let x: X<number> = { a: 1 };
let y: X<string> = { a: 'a' };
x === y; // Invalid as the type argument is used in the final structure

interface X<T> {}
const x: X<number> = 1;
const y: X<string> = 'a';
x === y; // Valid as the type argument is not used in the final structure
```

Jeśli argument typu generycznego nie został określony, wszystkie nieokreślone argumenty są traktowane jako typy any:

```
type X = <T>(x: T) => T;
type Y = <K>(y: K) => K;
let x: X = x => x;
let y: Y = y => y;
x = y; // Valid
```

Zapamiętaj:

```
let a: number = 1;
let b: number = 2;
a = b; // Valid, everything is assignable to itself

let c: any;
c = 1; // Valid, all types are assignable to any

let d: unknown;
d = 1; // Valid, all types are assignable to unknown

let e: unknown;
let e1: unknown = e; // Valid, unknown is only assignable to
itself and any
let e2: any = e; // Valid
let e3: number = e; // Invalid

let f: never;
f = 1; // Invalid, nothing is assignable to never

let g: void;
let g1: any;
g = 1; // Invalid, void is not assignable to or from anything
except any
g = g1; // Valid
```

Należy pamiętać, że gdy opcja „strictNullChecks” jest włączona, wartości „null” i „undefined” są traktowane podobnie do „void”;

w przeciwnym razie są traktowane podobnie do „never”.

## Typy jako zbiory

W TypeScript typ jest zbiorem możliwych wartości. Zbiór ten jest również nazywany dziedziną typu. Każdą wartość danego typu można postrzegać jako element zbioru. Typ określa ograniczenia, które musi spełniać każdy element zbioru, aby można go było uznać za należący do tego zbioru. Podstawowym zadaniem TypeScript jest sprawdzanie i weryfikowanie, czy jeden zbiór jest podzbiorem innego.

TypeScript obsługuje różne rodzaje zbiorów:

| **Pojęcie z teorii zbiorów** | **TypeScript** | **Uwagi** |
|---|---|---|
| Zbiór pusty | never | „never” nie zawiera niczego poza samym sobą |
| Zbiór jednoelementowy | undefined / null / typ literałowy | |
| Zbiór skończony | boolean / unia | |
| Zbiór nieskończony | string / number / object | |
| Zbiór uniwersalny | any / unknown | Każdy element należy do „any” i każdy zbiór jest jego podzbiorem / „unknown” jest bezpiecznym typowo odpowiednikiem „any” |

Oto kilka przykładów:

| **TypeScript** | **Pojęcie z teorii zbiorów** | **Przykład** |
|---|---|---|
| never | ∅ (zbiór pusty) | const x: never = ‘x’; // Error: Type ‘string’ is not assignable to type ‘never’ |

| | | |
|---|---|---|
| Typ literałowy | Zbiór jednoelementowy | type X = ‘X’;<br>type Y = 7; |
| Wartość przypisywalna do T | Wartość ∈ T (należy do) | type XY = ‘X’ \| ‘Y’;<br>const x: XY = ‘X’; |
| T1 przypisywalny do T2 | T1 ⊆ T2 (podzbiór) | type XY = ‘X’ \| ‘Y’;<br>const x: XY = ‘X’;<br>const j: XY = ‘J’; // Type “‘J”’ is not assignable to type ‘XY’. |
| T1 extends T2 | T1 ⊆ T2 (podzbiór) | type X = ‘X’ extends string ? true : false; |
| T1 \| T2 | T1 ∪ T2 (unia) | type XY = ‘X’ \| ‘Y’;<br>type JK = 1 \| 2; |
| T1 & T2 | T1 ∩ T2 (przecięcie) | type X = { a: string }<br>type Y = { b: string }<br>type XY = X & Y<br>const x: XY = { a: ‘a’, b: ‘b’ } |
| unknown | Zbiór uniwersalny | const x: unknown = 1 |

## Unia (T1 | T2) tworzy szerszy zbiór (obejmujący oba typy):

```
type X = {
    a: string;
};
type Y = {
    b: string;
};
type XY = X | Y;
const r: XY = { a: 'a', b: 'x' }; // Valid
```

Przecięcie (T1 & T2) tworzy węższy zbiór (tylko część wspólną):

```
type X = {
    a: string;
};
type Y = {
    a: string;
    b: string;
};
type XY = X & Y;
const r: XY = { a: 'a' }; // Invalid
const j: XY = { a: 'a', b: 'b' }; // Valid
```

Słowo kluczowe `extends` można w tym kontekście rozumieć jako „jest podzbiorem”. Nakłada ono ograniczenie na typ. Gdy `extends` jest używane z typem generycznym, ogranicza parametr typu generycznego do bardziej szczegółowego typu.

Należy pamiętać, że `extends` nie ma tutaj nic wspólnego z dziedziczeniem klas w rozumieniu programowania obiektowego.

TypeScript korzysta z typów strukturalnych i nie ma ścisłej hierarchii nominalnej. Jak pokazuje poniższy przykład, dwa typy mogą się nakładać, mimo że żaden z nich nie jest podtypem drugiego, ponieważ TypeScript bierze pod uwagę strukturę, czyli kształt obiektów.

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
interface Z extends Y {
    c: string;
```

```
}
const z: Z = { a: 'a', b: 'b', c: 'c' };
interface X1 {
    a: string;
}
interface Y1 {
    a: string;
    b: string;
}
interface Z1 {
    a: string;
    b: string;
    c: string;
}
const z1: Z1 = { a: 'a', b: 'b', c: 'c' };

const r: Z1 = z; // Valid
```

## Przypisywanie typu: deklaracje typów i asercje typów

W TypeScript typ można przypisać na różne sposoby:

### Deklaracja typu

W poniższym przykładzie używamy x: X („: Type”), aby zadeklarować typ zmiennej x.

```
type X = {
    a: string;
};

// Type declaration
const x: X = {
    a: 'a',
};
```

Jeśli zmienna nie ma określonego formatu, TypeScript zgłosi błąd. Na przykład:

```
type X = {
    a: string;
};

const x: X = {
    a: 'a',
     b: 'b', // Error: Object literal may only specify known
properties
};
```

### Asercja typu

Asercję można dodać za pomocą słowa kluczowego `as`. Informuje to kompilator, że programista ma więcej informacji o typie i wycisza wszelkie błędy, które mogą wystąpić.

Na przykład:

```
type X = {
    a: string;
};
const x = {
    a: 'a',
    b: 'b',
} as X;
```

W powyższym przykładzie za pomocą słowa kluczowego as określono, że obiekt x ma typ X. Informuje to kompilator TypeScript, że obiekt jest zgodny z określonym typem, mimo że ma dodatkową właściwość b, której nie ma w definicji typu.

Asercje typów są przydatne w sytuacjach, w których trzeba określić bardziej szczegółowy typ, zwłaszcza podczas pracy z

DOM. Na przykład:

```
const myInput = document.getElementById('my_input') as
HTMLInputElement;
```

W tym przypadku asercja typu as HTMLInputElement informuje TypeScript, że wynik getElementById należy traktować jako HTMLInputElement. Asercji typów można również używać do zmiany mapowania kluczy, jak pokazano w poniższym przykładzie z literałami szablonowymi:

```
type J<Type> = {
    [Property in keyof Type as `prefix_${string &
        Property}`]: () => Type[Property];
};
type X = {
    a: string;
    b: number;
};
type Y = J<X>;
```

W tym przykładzie typ `J<Type>` używa typu mapowanego z literałem szablonowym, aby ponownie zmapować klucze typu Type. Tworzy nowe właściwości z prefiksem „prefix_” dodanym do każdego klucza, a odpowiadające im wartości są funkcjami zwracającymi oryginalne wartości właściwości.

Warto zauważyć, że podczas używania asercji typu TypeScript nie przeprowadza kontroli nadmiarowych właściwości. Dlatego gdy struktura obiektu jest znana z wyprzedzeniem, zasadniczo lepiej jest użyć deklaracji typu.

### Deklaracje otoczenia

Deklaracje otoczenia to pliki opisujące typy kodu JavaScript; ich nazwy mają format `.d.ts`. Zazwyczaj są importowane i używane do opisywania typami istniejących bibliotek JavaScript lub do dodawania typów do istniejących plików JS w projekcie.

Typy dla wielu popularnych bibliotek można znaleźć pod adresem: https://github.com/DefinitelyTyped/DefinitelyTyped/

i zainstalować za pomocą polecenia:

```
npm install --save-dev @types/library-name
```

Własne deklaracje otoczenia można importować za pomocą odwołania z potrójnym ukośnikiem:

```
/// <reference path="./library-types.d.ts" />
```

Deklaracji otoczenia można używać nawet w plikach JavaScript za pomocą `// @ts-check`.

Słowo kluczowe `declare` umożliwia definiowanie typów dla istniejącego kodu JavaScript bez jego importowania, pełniąc funkcję symbolu zastępczego dla typów pochodzących z innego pliku lub dostępnych globalnie.

## Kontrola właściwości i kontrola nadmiarowych właściwości

TypeScript opiera się na strukturalnym systemie typów, ale kontrola nadmiarowych właściwości jest funkcją TypeScript, która pozwala sprawdzić, czy obiekt ma dokładnie właściwości określone w typie.

Kontrola nadmiarowych właściwości jest przeprowadzana na przykład podczas przypisywania literałów obiektowych do zmiennych lub przekazywania ich jako argumentów do nadmiarowej właściwości funkcji.

```
type X = {
    a: string;
};
const y = { a: 'a', b: 'b' };
const x: X = y; // Valid because structural typing
const w: X = { a: 'a', b: 'b' }; // Invalid because excess
property checking
```

## Typy słabe

Typ uznaje się za słaby, gdy zawiera wyłącznie zestaw właściwości opcjonalnych:

```
type X = {
    a?: string;
    b?: string;
};
```

TypeScript uznaje przypisanie czegokolwiek do typu słabego za błąd, jeśli nie występują żadne wspólne właściwości. Na przykład poniższy kod zgłasza błąd:

```
type Options = {
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;

fn({ c: 'c' }); // Invalid
```

Choć nie jest to zalecane, w razie potrzeby można pominąć tę kontrolę za pomocą asercji typu:

```
type Options = {
    a?: string;
    b?: string;
};
const fn = (options: Options) => undefined;
fn({ c: 'c' } as Options); // Valid
```

Można też dodać unknown do sygnatury indeksowej typu słabego:

```
type Options = {
    [prop: string]: unknown;
    a?: string;
    b?: string;
};

const fn = (options: Options) => undefined;
fn({ c: 'c' }); // Valid
```

## Ścisła kontrola literałów obiektowych (świeżość)

Ścisła kontrola literałów obiektowych, czasami nazywana „świeżością”, jest funkcją TypeScript, która pomaga wykrywać nadmiarowe lub błędnie zapisane właściwości, które w przeciwnym razie pozostałyby niezauważone podczas zwykłej kontroli typów strukturalnych.

Podczas tworzenia literału obiektowego kompilator TypeScript uznaje go za „świeży”. Jeśli literał obiektowy zostanie przypisany do zmiennej lub przekazany jako parametr, TypeScript zgłosi błąd, jeśli literał ten określa właściwości, które nie istnieją w typie docelowym.

„Świeżość” znika jednak, gdy literał obiektowy zostanie poszerzony lub zostanie użyta asercja typu.

Oto kilka przykładów ilustrujących tę funkcję:

```
type X = { a: string };
type Y = { a: string; b: string };

let x: X;
x = { a: 'a', b: 'b' }; // Freshness check: Invalid assignment
var y: Y;
y = { a: 'a', bx: 'bx' }; // Freshness check: Invalid assignment

const fn = (x: X) => console.log(x.a);

fn(x);
fn(y); // Widening: No errors, structurally type compatible

fn({ a: 'a', bx: 'b' }); // Freshness check: Invalid argument

let c: X = { a: 'a' };
let d: Y = { a: 'a', b: '' };
c = d; // Widening: No Freshness check
```

## Wnioskowanie typów

TypeScript może wnioskować typy, jeśli adnotacji nie podano podczas:

- Inicjalizacji zmiennej.
- Inicjalizacji elementu składowego.
- Ustawiania wartości domyślnych parametrów.
- Określania typu zwracanego przez funkcję.

Na przykład:

```
let x = 'x'; // The type inferred is string
```

Kompilator TypeScript analizuje wartość lub wyrażenie i określa jego typ na podstawie dostępnych informacji.

## Bardziej zaawansowane wnioskowanie

Gdy we wnioskowaniu typu używanych jest wiele wyrażeń, TypeScript szuka „najlepszego wspólnego typu". Na przykład:

```
let x = [1, 'x', 1, null]; // The type inferred is: (string | number | null)[]
```

Jeśli kompilator nie może znaleźć najlepszego wspólnego typu, zwraca typ unii. Na przykład:

```
let x = [new RegExp('x'), new Date()]; // Type inferred is: (RegExp | Date)[]
```

TypeScript wykorzystuje „typowanie kontekstowe" oparte na miejscu występowania zmiennej, aby wnioskować typy. W poniższym przykładzie kompilator wie, że `e` jest typu `MouseEvent`, ze względu na typ zdarzenia `click` zdefiniowany w pliku lib.d.ts, który zawiera deklaracje otoczenia dla różnych typowych konstrukcji JavaScript i DOM:

```
window.addEventListener('click', function (e) {}); // The inferred type of e is MouseEvent
```

## Poszerzanie typów

Poszerzanie typów jest procesem, w którym TypeScript przypisuje typ do zmiennej zainicjalizowanej bez adnotacji typu. Pozwala przechodzić od węższych typów do szerszych, ale nie odwrotnie. W poniższym przykładzie:

```
let x = 'x'; // TypeScript infers as string, a wide type
let y: 'y' | 'x' = 'y'; // y types is a union of literal types
y = x; // Invalid Type 'string' is not assignable to type '"x"
| "y"'.
```

TypeScript przypisuje typ `string` do `x` na podstawie pojedynczej wartości podanej podczas inicjalizacji (`x`). Jest to przykład poszerzania.

TypeScript zapewnia sposoby kontrolowania procesu poszerzania, na przykład przez użycie „const”.

## Const

Użycie słowa kluczowego `const` podczas deklarowania zmiennej powoduje, że TypeScript wnioskuje węższy typ.

Na przykład:

```
const x = 'x'; // TypeScript infers the type of x as 'x', a
narrower type
let y: 'y' | 'x' = 'y';
y = x; // Valid: The type of x is inferred as 'x'
```

Dzięki użyciu `const` do zadeklarowania zmiennej x jej typ zostaje zawężony do konkretnej wartości literałowej ‘x’. Ponieważ typ x jest zawężony, można przypisać go do zmiennej y bez żadnego błędu. Typ może zostać wywnioskowany w ten sposób, ponieważ zmiennym `const` nie można ponownie przypisać wartości, a więc ich typ można zawęzić do konkretnego typu literałowego, w tym przypadku do typu literałowego ‘x’.

### Modyfikator Const parametrów typu

Od wersji 5.0 w TypeScript można określić atrybut `const` dla parametru typu generycznego. Pozwala to wywnioskować możliwie najbardziej precyzyjny typ. Spójrzmy na przykład bez użycia `const`:

```
function identity<T>(value: T) {
    // No const here
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred
is: { a: string; b: string; }
```

Jak widać, właściwości `a` i `b` mają wywnioskowany typ `string`.

Zobaczmy teraz różnicę w wersji z `const`:

```
function identity<const T>(value: T) {
    // Using const modifier on type parameters
    return value;
}
const values = identity({ a: 'a', b: 'b' }); // Type inferred
is: { a: "a"; b: "b"; }
```

Teraz widać, że właściwości `a` i `b` są wnioskowane jako literały łańcuchowe, a nie tylko jako typy `string`.

### Asercja Const

Ta funkcja pozwala zadeklarować zmienną z bardziej precyzyjnym typem literałowym na podstawie jej wartości inicjalizacyjnej, wskazując kompilatorowi, że wartość powinna być traktowana jako niemutowalny literał. Oto kilka przykładów:

Dla pojedynczej właściwości:

```
const v = {
    x: 3 as const,
};
v.x = 3;
```

Dla całego obiektu:

```
const v = {
    x: 1,
    y: 2,
} as const;
```

Może to być szczególnie przydatne podczas definiowania typu krotki:

```
const x = [1, 2, 3]; // number[]
const y = [1, 2, 3] as const; // Tuple of readonly [1, 2, 3]
```

## Jawna adnotacja typu

Możemy jawnie przekazać konkretny typ. W poniższym przykładzie właściwość `x` ma typ `number`:

```
const v = {
    x: 1, // Inferred type: number (widening)
};
v.x = 3; // Valid
```

Adnotację typu można doprecyzować za pomocą unii typów literałowych:

```
const v: { x: 1 | 2 | 3 } = {
    x: 1, // x is now a union of literal types: 1 | 2 | 3
};
v.x = 3; // Valid
v.x = 100; // Invalid
```

## Zawężanie typów

Zawężanie typów jest w TypeScript procesem, w którym typ ogólny zostaje zawężony do bardziej szczegółowego typu. Dzieje się tak, gdy TypeScript analizuje kod i ustala, że określone warunki lub operacje mogą doprecyzować informacje o typie.

Typy można zawężać na różne sposoby, w tym przez:

### Warunki

Za pomocą instrukcji warunkowych, takich jak `if` lub `switch`, TypeScript może zawęzić typ na podstawie wyniku warunku. Na przykład:

```
let x: number | undefined = 10;

if (x !== undefined) {
    x += 100; // The type is number, which had been narrowed by the condition
}
```

### Zgłoszenie wyjątku lub zwrócenie wartości

Zgłoszenie błędu lub wcześniejsze zwrócenie wartości z gałęzi może sprawić, że TypeScript zawęzi typ. Na przykład:

```
let x: number | undefined = 10;

if (x === undefined) {
    throw 'error';
}
x += 100;
```

Inne sposoby zawężania typów w TypeScript obejmują:

- Operator `instanceof`: służy do sprawdzania, czy obiekt jest instancją określonej klasy.
- Operator `in`: służy do sprawdzania, czy właściwość istnieje w obiekcie.
- Operator `typeof`: służy do sprawdzania typu wartości w czasie wykonywania.
- Wbudowane funkcje, takie jak `Array.isArray()`: służą do sprawdzania, czy wartość jest tablicą.

### Unia dyskryminowana

Użycie „unii dyskryminowanej" jest w TypeScript wzorcem, w którym do obiektów dodaje się jawny „znacznik", aby rozróżnić różne typy w obrębie unii. Wzorzec ten jest również nazywany „unią znakowaną". W poniższym przykładzie „znacznik" jest reprezentowany przez właściwość „type":

```
type A = { type: 'type_a'; value: number };
type B = { type: 'type_b'; value: string };

const x = (input: A | B): string | number => {
    switch (input.type) {
        case 'type_a':
            return input.value + 100; // type is A
        case 'type_b':
            return input.value + 'extra'; // type is B
    }
};
```

### Typy strażnicze zdefiniowane przez użytkownika

W przypadkach, gdy TypeScript nie jest w stanie określić typu, można napisać funkcję pomocniczą znaną jako „typ strażniczy zdefiniowany przez użytkownika”. W poniższym przykładzie użyjemy predykatu typu, aby zawęzić typ po zastosowaniu określonego filtrowania:

```
const data = ['a', null, 'c', 'd', null, 'f'];

const r1 = data.filter(x => x != null); // The type is (string
| null)[], TypeScript was not able to infer the type properly

const isValid = (item: string | null): item is string => item
!== null; // Custom type guard

const r2 = data.filter(isValid); // The type is fine now
string[], by using the predicate type guard we were able to
narrow the type
```

### Zawężanie za pomocą switch-true

TypeScript 5.3 dodaje zawężanie za pomocą switch-true, które umożliwia zastąpienie nieczytelnych łańcuchów if/else konstrukcją switch (true) wykorzystującą warunki logiczne. Poprawia to czytelność, a jednocześnie nadal zawęża typy. Przypomina dopasowywanie wzorców, ale jest prostsze.

```
function classify(x: unknown) {
    switch (true) {
        case typeof x === 'string':
            return `"${x.toUpperCase()}"`;
        case typeof x === 'number':
            return x > 0 ? 'positive' : 'negative';
        case Array.isArray(x):
            return `[${x.length} items]`;
        default:
            return 'something else';
```

```
    }
}
```

# Typy pierwotne

TypeScript obsługuje 7 typów pierwotnych. Pierwotny typ danych to typ, który nie jest obiektem i nie ma powiązanych z nim żadnych metod. W TypeScript wszystkie typy pierwotne są niezmienne, co oznacza, że po przypisaniu ich wartości nie można ich zmienić.

## string

Typ pierwotny `string` przechowuje dane tekstowe, a jego wartość jest zawsze ujęta w cudzysłowy podwójne lub pojedyncze.

```
const x: string = 'x';
const y: string = 'y';
```

Ciągi znaków mogą obejmować wiele wierszy, jeśli są otoczone znakiem grawisu (`):

```
let sentence: string = `xxx,
   yyy`;
```

## boolean

Typ danych `boolean` w TypeScript przechowuje wartość binarną: `true` albo `false`.

```
const isReady: boolean = true;
```

## number

Typ danych `number` w TypeScript jest reprezentowany przez 64-bitową wartość zmiennoprzecinkową. Typ `number` może reprezentować liczby całkowite i ułamkowe. TypeScript obsługuje również liczby szesnastkowe, binarne i ósemkowe, na przykład:

```
const decimal: number = 10;
const hexadecimal: number = 0xa00d; // Hexadecimal starts with
0x
const binary: number = 0b1010; // Binary starts with 0b
const octal: number = 0o633; // Octal starts with 0o
```

## bigint

Typ `bigint` reprezentuje wartości całkowite, które mogą być większe niż maksymalna bezpieczna liczba całkowita obsługiwana przez `number`, czyli 2^53 - 1.

Wartość `bigint` można utworzyć, wywołując wbudowaną funkcję `BigInt()` lub dodając `n` na końcu dowolnego literału liczby całkowitej:

```
const x: bigint = BigInt(9007199254740991);
const y: bigint = 9007199254740991n;
```

Uwagi:

- Wartości `bigint` nie można mieszać z wartościami `number` ani używać ich z wbudowanym obiektem `Math`; należy je przekonwertować do tego samego typu.
- Wartości `bigint` są dostępne tylko wtedy, gdy konfiguracja elementu docelowego to ES2020 lub nowsza wersja.

## Symbol

Symbole są unikatowymi identyfikatorami, których można używać jako kluczy właściwości w obiektach, aby zapobiegać konfliktom nazw.

```
type Obj = {
    [sym: symbol]: number;
};

const a = Symbol('a');
const b = Symbol('b');
let obj: Obj = {};
obj[a] = 123;
obj[b] = 456;

console.log(obj[a]); // 123
console.log(obj[b]); // 456
```

## null i undefined

Typy `null` i `undefined` reprezentują brak wartości.

Typ `undefined` oznacza, że wartość nie została przypisana ani zainicjalizowana lub wskazuje na niezamierzony brak wartości.

Typ `null` oznacza, że wiemy, iż pole nie ma wartości, więc wartość jest niedostępna i wskazuje na zamierzony brak wartości.

## Tablica

Tablica (`array`) to typ danych, który może przechowywać wiele wartości tego samego typu lub różnych typów. Można ją zdefiniować przy użyciu następującej składni:

```
const x: string[] = ['a', 'b'];
const y: Array<string> = ['a', 'b'];
```

```
const j: Array<string | number> = ['a', 1, 'b', 2]; // Union
```

TypeScript obsługuje tablice tylko do odczytu przy użyciu następującej składni:

```
const x: readonly string[] = ['a', 'b']; // Readonly modifier
const y: ReadonlyArray<string> = ['a', 'b'];
const j: ReadonlyArray<string | number> = ['a', 1, 'b', 2];
j.push('x'); // Invalid
```

TypeScript obsługuje krotki i krotki tylko do odczytu:

```
const x: [string, number] = ['a', 1];
const y: readonly [string, number] = ['a', 1];
```

## any

Typ danych any reprezentuje dosłownie „dowolną” wartość i jest typem domyślnym, gdy TypeScript nie może wywnioskować typu lub gdy typ nie został określony.

Podczas używania any kompilator TypeScript pomija sprawdzanie typów, dlatego użycie any nie zapewnia bezpieczeństwa typów. Zasadniczo nie należy używać any do wyciszania kompilatora, gdy wystąpi błąd. Zamiast tego należy skupić się na naprawieniu błędu, ponieważ użycie any umożliwia naruszanie kontraktów i utratę korzyści z autouzupełniania w TypeScript.

Typ any może być przydatny podczas stopniowej migracji z JavaScript do TypeScript, ponieważ pozwala wyciszyć kompilator.

W nowych projektach należy używać opcji konfiguracji TypeScript noImplicitAny, dzięki której TypeScript zgłasza błędy

w miejscach, gdzie użyto lub wywnioskowano `any`.

Typ `any` jest zwykle źródłem błędów, które mogą maskować rzeczywiste problemy z typami. Należy unikać jego używania, gdy tylko jest to możliwe.

## Adnotacje typów

Do zmiennych zadeklarowanych za pomocą `var`, `let` i `const` można opcjonalnie dodać typ:

```
const x: number = 1;
```

TypeScript dobrze radzi sobie z wnioskowaniem typów, zwłaszcza tych prostych, dlatego w większości przypadków takie deklaracje nie są konieczne.

Do parametrów funkcji można dodać adnotacje typów:

```
function sum(a: number, b: number) {
    return a + b;
}
```

Poniżej znajduje się przykład użycia funkcji anonimowej (nazywanej również funkcją lambda):

```
const sum = (a: number, b: number) => a + b;
```

Tych adnotacji można uniknąć, gdy parametr ma wartość domyślną:

```
const sum = (a = 10, b: number) => a + b;
```

Do funkcji można dodać adnotacje typu zwracanego:

```
const sum = (a = 10, b: number): number => a + b;
```

Jest to szczególnie przydatne w przypadku bardziej złożonych funkcji, ponieważ zapisanie typu zwracanego przed implementacją może pomóc w przemyśleniu funkcji.

Zasadniczo warto dodawać adnotacje do sygnatur typów, ale nie do zmiennych lokalnych w ciele funkcji oraz zawsze dodawać typy do literałów obiektowych.

# Właściwości opcjonalne

Obiekt może określać właściwości opcjonalne przez dodanie znaku zapytania `?` na końcu nazwy właściwości:

```
type X = {
    a: number;
    b?: number; // Optional
};
```

Gdy właściwość jest opcjonalna, można określić dla niej wartość domyślną:

```
type X = {
    a: number;
    b?: number;
};
const x = ({ a, b = 100 }: X) => a + b;
```

# Właściwości tylko do odczytu

Można uniemożliwić zapisywanie właściwości za pomocą modyfikatora `readonly`, który gwarantuje, że właściwości nie można ponownie zapisać, ale nie zapewnia całkowitej niezmienności:

```ts
interface Y {
    readonly a: number;
}

type X = {
    readonly a: number;
};

type J = Readonly<{
    a: number;
}>;

type K = {
    readonly [index: number]: string;
};
```

# Sygnatury indeksowe

W TypeScript można używać typów `string`, `number` i `symbol` jako sygnatur indeksowych:

```ts
type K = {
    [name: string | number]: string;
};
const k: K = { x: 'x', 1: 'b' };
console.log(k['x']);
console.log(k[1]);
console.log(k['1']); // Same result as k[1]
```

Należy pamiętać, że JavaScript automatycznie konwertuje indeks typu `number` na indeks typu `string`, dlatego `k[1]` i `k["1"]` zwracają tę samą wartość.

# Rozszerzanie typów

Można rozszerzyć `interface` (skopiować elementy składowe z innego typu):

```
interface X {
    a: string;
}
interface Y extends X {
    b: string;
}
```

Można również rozszerzać wiele typów:

```
interface A {
    a: string;
}
interface B {
    b: string;
}
interface Y extends A, B {
    y: string;
}
```

Słowo kluczowe `extends` działa tylko z interfejsami i klasami; w przypadku typów należy użyć przecięcia:

```
type A = {
    a: number;
};
type B = {
    b: number;
};
type C = A & B;
```

Można rozszerzyć typ za pomocą interfejsu, ale nie odwrotnie:

```
type A = {
    a: string;
};
```

```
interface B extends A {
    b: string;
}
```

# Typy literałowe

Typ literałowy jest zbiorem jednoelementowym należącym do typu zbiorczego; definiuje bardzo dokładną wartość, która jest typem pierwotnym JavaScript.

Typami literałowymi w TypeScript są liczby, ciągi znaków i wartości logiczne.

Przykład literałów:

```
const a = 'a'; // String literal type
const b = 1; // Numeric literal type
const c = true; // Boolean literal type
```

Literałowe typy ciągów znaków, liczb i wartości logicznych są używane w uniach, strażnikach typów i aliasach typów. W poniższym przykładzie można zobaczyć alias typu unii. `O` składa się wyłącznie z określonych wartości; żaden inny ciąg znaków nie jest prawidłowy:

```
type O = 'a' | 'b' | 'c';
```

# Wnioskowanie typów literałowych

Wnioskowanie typów literałowych to funkcja TypeScript, która pozwala wywnioskować typ zmiennej lub parametru na podstawie jego wartości.

W poniższym przykładzie widać, że TypeScript traktuje `x` jako typ literałowy, ponieważ jego wartości nie można później zmienić, natomiast `y` jest wnioskowane jako `string`, ponieważ można je później zmodyfikować.

```
const x = 'x'; // Literal type of 'x', because this value cannot be changed
let y = 'y'; // Type string, as we can change this value
```

W poniższym przykładzie widać, że `o.x` zostało wywnioskowane jako `string` (a nie literał `a`), ponieważ TypeScript zakłada, że wartość może zostać później zmieniona.

```
type X = 'a' | 'b';

let o = {
    x: 'a', // This is a wider string
};

const fn = (x: X) => `${x}-foo`;

console.log(fn(o.x)); // Argument of type 'string' is not assignable to parameter of type 'X'
```

Jak widać, kod zgłasza błąd podczas przekazywania `o.x` do `fn`, ponieważ X jest węższym typem.

Ten problem można rozwiązać, używając asercji typu z `const` lub typem `X`:

```
let o = {
    x: 'a' as const,
};
```

lub:

```
let o = {
    x: 'a' as X,
};
```

# strictNullChecks

`strictNullChecks` to opcja kompilatora TypeScript wymuszająca ścisłe sprawdzanie wartości null. Gdy ta opcja jest włączona, do zmiennych i parametrów można przypisać `null` lub `undefined` tylko wtedy, gdy zostały jawnie zadeklarowane jako należące do tego typu przy użyciu typu unii `null | undefined`. Jeśli zmienna lub parametr nie zostały jawnie zadeklarowane jako dopuszczające null, TypeScript wygeneruje błąd, aby zapobiec potencjalnym błędom w czasie wykonywania.

# Wyliczenia

W TypeScript `enum` jest zbiorem nazwanych wartości stałych.

```
enum Color {
    Red = '#ff0000',
    Green = '#00ff00',
    Blue = '#0000ff',
}
```

Wyliczenia można definiować na różne sposoby:

## Wyliczenia liczbowe

W TypeScript wyliczenie liczbowe to `enum`, w którym każdej stałej przypisana jest wartość liczbowa, domyślnie zaczynająca się od 0.

```
enum Size {
    Small, // value starts from 0
    Medium,
    Large,
}
```

Można określić niestandardowe wartości przez ich jawne przypisanie:

```
enum Size {
    Small = 10,
    Medium,
    Large,
}
console.log(Size.Medium); // 11
```

## Wyliczenia ciągów znaków

W TypeScript wyliczenie ciągów znaków to `enum`, w którym każdej stałej przypisana jest wartość typu `string`.

```
enum Language {
    English = 'EN',
    Spanish = 'ES',
}
```

Uwaga: TypeScript pozwala na używanie heterogenicznych wyliczeń, w których elementy typu `string` i `number` mogą współistnieć.

## Wyliczenia stałe

Wyliczenie stałe w TypeScript to specjalny typ `enum`, którego wszystkie wartości są znane w czasie kompilacji i wstawiane bezpośrednio wszędzie tam, gdzie używane jest wyliczenie, co zapewnia wydajniejszy kod.

```
const enum Language {
    English = 'EN',
    Spanish = 'ES',
}
console.log(Language.English);
```

Zostanie skompilowane do:

```
console.log('EN' /* Language.English */);
```

Uwagi: Wartości wyliczeń stałych są zakodowane na stałe, co powoduje usunięcie wyliczenia i może zapewnić większą wydajność w samodzielnych bibliotekach, ale na ogół nie jest pożądane. Ponadto wyliczenia stałe nie mogą zawierać elementów wyliczanych.

## Mapowanie odwrotne

W TypeScript mapowanie odwrotne w wyliczeniach oznacza możliwość pobrania nazwy elementu wyliczenia na podstawie jego wartości. Domyślnie elementy wyliczenia mają mapowanie w przód z nazwy na wartość, ale mapowanie odwrotne można utworzyć przez jawne ustawienie wartości każdego elementu. Mapowania odwrotne są przydatne, gdy trzeba wyszukać element wyliczenia na podstawie jego wartości lub przejść po wszystkich elementach wyliczenia. Należy pamiętać, że mapowania odwrotne są generowane tylko dla liczbowych elementów wyliczenia, natomiast dla elementów typu `string` nie są generowane w ogóle.

Następujące wyliczenie:

```
enum Grade {
    A = 90,
    B = 80,
```

```
    C = 70,
    F = 'fail',
}
```

Jest kompilowane do:

```
'use strict';
var Grade;
(function (Grade) {
    Grade[(Grade['A'] = 90)] = 'A';
    Grade[(Grade['B'] = 80)] = 'B';
    Grade[(Grade['C'] = 70)] = 'C';
    Grade['F'] = 'fail';
})(Grade || (Grade = {}));
```

Dlatego mapowanie wartości na klucze działa dla liczbowych elementów wyliczenia, ale nie dla elementów typu `string`:

```
enum Grade {
    A = 90,
    B = 80,
    C = 70,
    F = 'fail',
}
const myGrade = Grade.A;
console.log(Grade[myGrade]); // A
console.log(Grade[90]); // A

const failGrade = Grade.F;
console.log(failGrade); // fail
console.log(Grade[failGrade]);  //  Element  implicitly  has  an
'any' type because index expression is not of type 'number'.
```

## Wyliczenia ambientowe

Wyliczenie ambientowe w TypeScript to rodzaj `enum` zdefiniowanego w pliku deklaracji (*.d.ts) bez powiązanej implementacji. Pozwala ono zdefiniować zestaw nazwanych

stałych, których można używać w sposób bezpieczny pod względem typów w różnych plikach bez konieczności importowania szczegółów implementacji do każdego pliku.

## Elementy wyliczane i stałe

W TypeScript element wyliczany to element `enum`, którego wartość jest obliczana w czasie wykonywania, natomiast element stały to element, którego wartość jest ustawiana w czasie kompilacji i nie może zostać zmieniona w czasie wykonywania. Elementy wyliczane są dozwolone w zwykłych wyliczeniach, natomiast elementy stałe są dozwolone zarówno w zwykłych wyliczeniach, jak i w `const enum`.

```
// Constant members
enum Color {
    Red = 1,
    Green = 5,
    Blue = Red + Green,
}
console.log(Color.Blue); // 6 generation at compilation time

// Computed members
enum Color {
    Red = 1,
    Green = Math.pow(2, 2),
    Blue = Math.floor(Math.random() * 3) + 1,
}
console.log(Color.Blue); // random number generated at run time
```

Wyliczenia są reprezentowane przez unie złożone z typów ich elementów. Wartości każdego elementu można określić za pomocą wyrażeń stałych lub niestałych, przy czym elementom o wartościach stałych przypisywane są typy literałowe. Dla zobrazowania rozważmy deklarację typu E i jego podtypów E.A,

E.B oraz E.C. W tym przypadku E reprezentuje unię E.A | E.B | E.C.

```
const identity = (value: number) => value;

enum E {
    A = 2 * 5, // Numeric literal
    B = 'bar', // String literal
    C = identity(42), // Opaque computed
}

console.log(E.C); //42
```

# Zawężanie typów

Zawężanie typów w TypeScript to proces uściślania typu zmiennej wewnątrz bloku warunkowego. Jest to przydatne podczas pracy z typami unii, gdy zmienna może mieć więcej niż jeden typ.

TypeScript rozpoznaje kilka sposobów zawężania typu:

## Strażniki typów typeof

Strażnik typu `typeof` to szczególny strażnik typów w TypeScript, który sprawdza typ zmiennej na podstawie jej wbudowanego typu JavaScript.

```
const fn = (x: number | string) => {
    if (typeof x === 'number') {
        return x + 1; // x is number
    }
    return -1;
};
```

## Zawężanie na podstawie prawdziwości

Zawężanie na podstawie prawdziwości w TypeScript polega na sprawdzeniu, czy zmienna ma wartość prawdziwą (`truthy`) czy fałszywą (`falsy`), aby odpowiednio zawęzić jej typ.

```
const toUpperCase = (name: string | null) => {
    if (name) {
        return name.toUpperCase();
    } else {
        return null;
    }
};
```

## Zawężanie na podstawie równości

Zawężanie na podstawie równości w TypeScript polega na sprawdzeniu, czy zmienna jest równa określonej wartości, aby odpowiednio zawęzić jej typ.

Jest używane w połączeniu z instrukcjami `switch` oraz operatorami równości, takimi jak `===`, `!==`, `==` i `!=`, w celu zawężania typów.

```
const checkStatus = (status: 'success' | 'error') => {
    switch (status) {
        case 'success':
            return true;
        case 'error':
            return null;
    }
};
```

## Zawężanie za pomocą operatora in

Zawężanie za pomocą operatora `in` w TypeScript to sposób zawężania typu zmiennej na podstawie tego, czy właściwość istnieje w typie zmiennej.

```
type Dog = {
    name: string;
    breed: string;
};

type Cat = {
    name: string;
    likesCream: boolean;
};

const getAnimalType = (pet: Dog | Cat) => {
    if ('breed' in pet) {
        return 'dog';
    } else {
        return 'cat';
    }
};
```

## Zawężanie za pomocą instanceof

Zawężanie za pomocą operatora `instanceof` w TypeScript to sposób zawężania typu zmiennej na podstawie jej funkcji konstruktora poprzez sprawdzenie, czy obiekt jest instancją określonej klasy lub interfejsu.

```
class Square {
    constructor(public width: number) {}
}
class Rectangle {
    constructor(
        public width: number,
        public height: number
    ) {}
```

```
}
function area(shape: Square | Rectangle) {
    if (shape instanceof Square) {
        return shape.width * shape.width;
    } else {
        return shape.width * shape.height;
    }
}
const square = new Square(5);
const rectangle = new Rectangle(5, 10);
console.log(area(square)); // 25
console.log(area(rectangle)); // 50
```

# Przypisania

Zawężanie typów za pomocą przypisań w TypeScript to sposób zawężania typu zmiennej na podstawie przypisanej do niej wartości. Gdy do zmiennej zostanie przypisana wartość, TypeScript wnioskuje jej typ na podstawie przypisanej wartości i zawęża typ zmiennej tak, aby odpowiadał wywnioskowanemu typowi.

```
let value: string | number;
value = 'hello';
if (typeof value === 'string') {
    console.log(value.toUpperCase());
}
value = 42;
if (typeof value === 'number') {
    console.log(value.toFixed(2));
}
```

# Analiza przepływu sterowania

Analiza przepływu sterowania w TypeScript to sposób statycznej analizy przepływu kodu w celu wywnioskowania

typów zmiennych, co pozwala kompilatorowi zawężać typy tych zmiennych w razie potrzeby na podstawie wyników analizy.

Przed TypeScript 4.4 analiza przepływu kodu była stosowana tylko do kodu wewnątrz instrukcji `if`, ale od TypeScript 4.4 może być również stosowana do wyrażeń warunkowych i dostępów do właściwości dyskryminujących, do których pośrednio odwołują się zmienne `const`.

Na przykład:

```
const f1 = (x: unknown) => {
    const isString = typeof x === 'string';
    if (isString) {
        x.length;
    }
};

const f2 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    if (isFoo) {
        obj.foo;
    } else {
        obj.bar;
    }
};
```

Przykłady, w których zawężanie nie zachodzi:

```
const f1 = (x: unknown) => {
    let isString = typeof x === 'string';
    if (isString) {
```

```
        x.length; // Error, no narrowing because isString it is
not const
    }
};

const f6 = (
     obj: { kind: 'foo'; foo: string } | { kind: 'bar'; bar:
number }
) => {
    const isFoo = obj.kind === 'foo';
    obj = obj;
    if (isFoo) {
        obj.foo; // Error, no narrowing because obj is assigned
in function body
    }
};
```

Uwaga: W wyrażeniach warunkowych analizowanych jest do pięciu poziomów pośrednictwa.

# Predykaty typów

Predykaty typów w TypeScript to funkcje zwracające wartość logiczną, które służą do zawężania typu zmiennej do bardziej szczegółowego typu.

```
const isString = (value: unknown): value is string => typeof
value === 'string';

const foo = (bar: unknown) => {
    if (isString(bar)) {
        console.log(bar.toUpperCase());
    } else {
        console.log('not a string');
    }
};
```

TypeScript 5.5 automatycznie wnioskuje predykaty typów (takie jak `x is T`) w funkcjach takich jak `.filter`, dzięki czemu wie, kiedy wartości takie jak `undefined` są usuwane — zapewnia to precyzyjniejsze typy i mniej błędów. Działa to w przypadku jednoznacznych sprawdzeń (np. `x !== undefined`), ale nie w przypadku niejednoznacznych sprawdzeń, takich jak `!!x`.

```
const nums = [1, null, 2].filter(x => x !== null);
```

# Unie dyskryminowane

Unie dyskryminowane w TypeScript są rodzajem typu unii, który wykorzystuje wspólną właściwość, zwaną dyskryminantem, do zawężania zbioru możliwych typów w unii.

```
type Square = {
    kind: 'square'; // Discriminant
    size: number;
};

type Circle = {
    kind: 'circle'; // Discriminant
    radius: number;
};

type Shape = Square | Circle;

const area = (shape: Shape) => {
    switch (shape.kind) {
        case 'square':
            return Math.pow(shape.size, 2);
        case 'circle':
            return Math.PI * Math.pow(shape.radius, 2);
    }
};
```

```
const square: Square = { kind: 'square', size: 5 };
const circle: Circle = { kind: 'circle', radius: 2 };

console.log(area(square)); // 25
console.log(area(circle)); // 12.566370614359172
```

## Specjalny typ never

Gdy zmienna zostanie zawężona do typu, który nie może zawierać żadnych wartości, kompilator TypeScript wywnioskuje, że zmienna musi być typu `never`. Dzieje się tak, ponieważ typ `never` reprezentuje wartość, której nigdy nie można utworzyć.

```
const printValue = (val: string | number) => {
    if (typeof val === 'string') {
        console.log(val.toUpperCase());
    } else if (typeof val === 'number') {
        console.log(val.toFixed(2));
    } else {
            // val has type never here because it can never be
anything other than a string or a number
        const neverVal: never = val;
        console.log(`Unexpected value: ${neverVal}`);
    }
};
```

## Sprawdzanie kompletności

Sprawdzanie kompletności to funkcja TypeScript, która zapewnia obsługę wszystkich możliwych przypadków unii dyskryminowanej w instrukcji `switch` lub instrukcji `if`.

```
type Direction = 'up' | 'down';
```

```
const move = (direction: Direction) => {
    switch (direction) {
        case 'up':
            console.log('Moving up');
            break;
        case 'down':
            console.log('Moving down');
            break;
        default:
            const exhaustiveCheck: never = direction;
                console.log(exhaustiveCheck); // This line will
never be executed
    }
};
```

Typ `never` służy do zapewnienia kompletności przypadku domyślnego oraz do zagwarantowania, że TypeScript zgłosi błąd, jeśli do typu Direction zostanie dodana nowa wartość bez jej obsługi w instrukcji `switch`.

# Typy obiektowe

W TypeScript typy obiektowe opisują strukturę obiektu. Określają nazwy i typy właściwości obiektu, a także to, czy właściwości te są wymagane, czy opcjonalne.

W TypeScript typy obiektowe można definiować na dwa podstawowe sposoby:

Interfejs definiuje strukturę obiektu przez określenie nazw, typów i opcjonalności jego właściwości.

```
interface User {
    name: string;
    age: number;
```

```
    email?: string;
}
```

Alias typu, podobnie jak interfejs, definiuje strukturę obiektu. Może jednak również utworzyć nowy typ niestandardowy oparty na istniejącym typie lub kombinacji istniejących typów. Obejmuje to definiowanie typów unii, typów przecięcia i innych typów złożonych.

```
type Point = {
    x: number;
    y: number;
};
```

Można również zdefiniować typ anonimowo:

```
const sum = (x: { a: number; b: number }) => x.a + x.b;
console.log(sum({ a: 5, b: 1 }));
```

# Typ krotki (anonimowy)

Typ krotki to typ reprezentujący tablicę o stałej liczbie elementów i odpowiadających im typach. Typ krotki wymusza określoną liczbę elementów oraz ich odpowiednie typy w stałej kolejności. Typy krotek są przydatne, gdy trzeba reprezentować kolekcję wartości o określonych typach, w której pozycja każdego elementu w tablicy ma konkretne znaczenie.

```
type Point = [number, number];
```

# Nazwany typ krotki (z etykietami)

Typy krotek mogą zawierać opcjonalne etykiety lub nazwy dla każdego elementu. Etykiety te służą zwiększeniu czytelności i

wspomaganiu narzędzi, ale nie wpływają na operacje, które można wykonywać na krotkach.

```
type T = string;
type Tuple1 = [T, T];
type Tuple2 = [a: T, b: T];
type Tuple3 = [a: T, T]; // Named Tuple plus Anonymous Tuple
```

# Krotka o stałej długości

Krotka o stałej długości to szczególny rodzaj krotki, który wymusza stałą liczbę elementów określonych typów i zabrania modyfikowania długości krotki po jej zdefiniowaniu.

Krotki o stałej długości są przydatne, gdy trzeba reprezentować kolekcję wartości o określonej liczbie elementów i określonych typach oraz zapewnić, że długość i typy krotki nie zostaną przypadkowo zmienione.

```
const x = [10, 'hello'] as const;
x.push(2); // Error
```

# Typ unii

Typ unii to typ reprezentujący wartość, która może być jednym z kilku typów. Typy unii zapisuje się za pomocą symbolu `|` między każdym możliwym typem.

```
let x: string | number;
x = 'hello'; // Valid
x = 123; // Valid
```

# Typy przecięcia

Typ przecięcia to typ reprezentujący wartość, która ma wszystkie właściwości co najmniej dwóch typów. Typy przecięcia zapisuje się za pomocą symbolu `&` między poszczególnymi typami.

```
type X = {
    a: string;
};

type Y = {
    b: string;
};

type J = X & Y; // Intersection

const j: J = {
    a: 'a',
    b: 'b',
};
```

# Indeksowanie typów

Indeksowanie typów oznacza możliwość definiowania typów, które można indeksować za pomocą klucza nieznanego z góry, przy użyciu sygnatury indeksowej do określenia typu właściwości, które nie zostały jawnie zadeklarowane.

```
type Dictionary<T> = {
    [key: string]: T;
};
const myDict: Dictionary<string> = { a: 'a', b: 'b' };
console.log(myDict['a']); // Returns a
```

# Typ na podstawie wartości

Typ na podstawie wartości w TypeScript oznacza automatyczne wnioskowanie typu z wartości lub wyrażenia za pomocą mechanizmu wnioskowania typów.

```
const x = 'x'; // TypeScript infers 'x' as a string literal with 'const' (immutable), but widens it to 'string' with 'let' (reassignable).
```

# Typ na podstawie wartości zwracanej przez funkcję

Typ na podstawie wartości zwracanej przez funkcję oznacza możliwość automatycznego wywnioskowania typu zwracanego przez funkcję na podstawie jej implementacji. Dzięki temu TypeScript może określić typ wartości zwracanej przez funkcję bez jawnych adnotacji typów.

```
const add = (x: number, y: number) => x + y; // TypeScript can infer that the return type of the function is a number
```

# Typ na podstawie modułu

Typ na podstawie modułu oznacza możliwość użycia wartości eksportowanych przez moduł do automatycznego wnioskowania ich typów. Gdy moduł eksportuje wartość określonego typu, TypeScript może użyć tej informacji do automatycznego wywnioskowania typu tej wartości po zaimportowaniu jej do innego modułu.

```
// calc.ts
export const add = (x: number, y: number) => x + y;
// index.ts
import { add } from 'calc';
const r = add(1, 2); // r is number
```

# Typy mapowane

Typy mapowane w TypeScript pozwalają tworzyć nowe typy na podstawie istniejącego typu przez przekształcenie każdej właściwości za pomocą funkcji mapującej. Mapując istniejące typy, można tworzyć nowe typy, które reprezentują te same informacje w innym formacie. Aby utworzyć typ mapowany, należy uzyskać dostęp do właściwości istniejącego typu za pomocą operatora `keyof`, a następnie zmienić je w celu utworzenia nowego typu. W poniższym przykładzie:

```
type MyMappedType<T> = {
    [P in keyof T]: T[P][];
};
type MyType = {
    foo: string;
    bar: number;
};
type MyNewType = MyMappedType<MyType>;
const x: MyNewType = {
    foo: ['hello', 'world'],
    bar: [1, 2, 3],
};
```

Definiujemy `MyMappedType`, aby mapował właściwości typu `T`, tworząc nowy typ, w którym każda właściwość jest tablicą swojego pierwotnego typu. Za jego pomocą tworzymy `MyNewType`, aby reprezentował te same informacje co `MyType`, ale z każdą właściwością w postaci tablicy.

# Modyfikatory typów mapowanych

Modyfikatory typów mapowanych w TypeScript umożliwiają przekształcanie właściwości w istniejącym typie:

- `readonly` lub `+readonly`: Powoduje, że właściwość w typie mapowanym jest tylko do odczytu.
- `-readonly`: Pozwala modyfikować właściwość w typie mapowanym.
- `?`: Oznacza właściwość w typie mapowanym jako opcjonalną.

Przykłady:

```
type ReadOnly<T> = { readonly [P in keyof T]: T[P] }; // All properties marked as read-only

type Mutable<T> = { -readonly [P in keyof T]: T[P] }; // All properties marked as mutable

type MyPartial<T> = { [P in keyof T]?: T[P] }; // All properties marked as optional
```

# Typy warunkowe

Typy warunkowe umożliwiają tworzenie typu zależnego od warunku, gdzie typ do utworzenia jest określany na podstawie wyniku warunku. Definiuje się je za pomocą słowa kluczowego `extends` i operatora trójargumentowego, aby warunkowo wybrać jeden z dwóch typów.

```
type IsArray<T> = T extends any[] ? true : false;

const myArray = [1, 2, 3];
const myNumber = 42;

type IsMyArrayAnArray = IsArray<typeof myArray>; // Type true
type IsMyNumberAnArray = IsArray<typeof myNumber>; // Type false
```

# Dystrybucyjne typy warunkowe

Dystrybucyjne typy warunkowe to funkcja, która umożliwia rozdzielenie typu na unię typów przez zastosowanie przekształcenia osobno do każdego elementu unii. Może to być szczególnie przydatne podczas pracy z typami mapowanymi lub typami wyższego rzędu.

```
type Nullable<T> = T extends any ? T | null : never;
type NumberOrBool = number | boolean;
type NullableNumberOrBool = Nullable<NumberOrBool>; // number |
boolean | null
```

# Wnioskowanie typu infer w typach warunkowych

Słowo kluczowe `infer` jest używane w typach warunkowych do wnioskowania (wyodrębniania) typu parametru generycznego z typu, który od niego zależy. Pozwala to pisać bardziej elastyczne definicje typów wielokrotnego użytku.

```
type ElementType<T> = T extends (infer U)[] ? U : never;
type Numbers = ElementType<number[]>; // number
type Strings = ElementType<string[]>; // string
```

# Predefiniowane typy warunkowe

W TypeScript predefiniowane typy warunkowe są wbudowanymi typami warunkowymi udostępnianymi przez język. Zostały zaprojektowane do wykonywania typowych przekształceń typów na podstawie cech danego typu.

`Exclude<UnionType, ExcludedType>`: Ten typ usuwa z Type wszystkie typy, które można przypisać do ExcludedType.

`Extract<Type, Union>`: Ten typ wyodrębnia z Union wszystkie typy, które można przypisać do Type.

`NonNullable<Type>`: Ten typ usuwa `null` i `undefined` z Type.

`ReturnType<Type>`: Ten typ wyodrębnia typ zwracany przez funkcję Type.

`Parameters<Type>`: Ten typ wyodrębnia typy parametrów funkcji Type.

`Required<Type>`: Ten typ sprawia, że wszystkie właściwości w Type są wymagane.

`Partial<Type>`: Ten typ sprawia, że wszystkie właściwości w Type są opcjonalne.

`Readonly<Type>`: Ten typ sprawia, że wszystkie właściwości w Type są tylko do odczytu.

## Szablonowe typy unii

Szablonowe typy unii mogą być używane do łączenia tekstu i manipulowania nim w systemie typów, na przykład:

```
type Status = 'active' | 'inactive';
type Products = 'p1' | 'p2';
type ProductId = `id-${Products}-${Status}`; // "id-p1-active"
| "id-p1-inactive" | "id-p2-active" | "id-p2-inactive"
```

## Typ any

Typ `any` jest specjalnym typem (uniwersalnym nadtypem), którego można użyć do reprezentowania wartości dowolnego typu (typów pierwotnych, obiektów, tablic, funkcji, błędów, symboli). Jest często używany w sytuacjach, gdy typ wartości nie jest znany w czasie kompilacji lub podczas pracy z wartościami pochodzącymi z zewnętrznych interfejsów API albo bibliotek, które nie mają typów TypeScript.

Używając typu `any`, informuje się kompilator TypeScript, że wartości powinny być reprezentowane bez żadnych ograniczeń. Aby zmaksymalizować bezpieczeństwo typów w kodzie, należy rozważyć następujące kwestie:

- Ograniczyć użycie `any` do konkretnych przypadków, w których typ jest rzeczywiście nieznany.
- Nie zwracać typów `any` z funkcji, ponieważ osłabia to bezpieczeństwo typów w kodzie, który ich używa.
- Zamiast `any` użyć `@ts-ignore`, jeśli trzeba wyciszyć kompilator.

```
let value: any;
value = true; // Valid
value = 7; // Valid
```

# Typ unknown

W TypeScript typ `unknown` reprezentuje wartość nieznanego typu. W przeciwieństwie do typu `any`, który dopuszcza wartość dowolnego typu, `unknown` wymaga sprawdzenia lub asercji typu przed użyciem wartości w określony sposób, dlatego na wartości `unknown` nie są dozwolone żadne operacje bez wcześniejszego zastosowania asercji lub zawężenia do bardziej szczegółowego typu.

Typ `unknown` można przypisać tylko do typów `any` i `unknown`; jest on bezpieczną pod względem typów alternatywą dla `any`.

```
let value: unknown;

let value1: unknown = value; // Valid
let value2: any = value; // Valid
let value3: boolean = value; // Invalid
let value4: number = value; // Invalid

const add = (a: unknown, b: unknown): number | undefined =>
     typeof a === 'number' && typeof b === 'number' ? a + b :
undefined;
console.log(add(1, 2)); // 3
console.log(add('x', 2)); // undefined
```

# Typ void

Typ `void` służy do wskazania, że funkcja nie zwraca wartości.

```
const sayHello = (): void => {
    console.log('Hello!');
};
```

# Typ never

Typ `never` reprezentuje wartości, które nigdy nie występują. Służy do oznaczania funkcji lub wyrażeń, które nigdy nie zwracają wartości albo zgłaszają błąd.

Na przykład nieskończona pętla:

```
const infiniteLoop = (): never => {
    while (true) {
        // do something
```

```
    }
};
```

Zgłaszanie błędu:

```
const throwError = (message: string): never => {
    throw new Error(message);
};
```

Typ `never` jest przydatny do zapewniania bezpieczeństwa typów i wykrywania potencjalnych błędów w kodzie. Pomaga TypeScript analizować i wnioskować bardziej precyzyjne typy, gdy jest używany w połączeniu z innymi typami i instrukcjami przepływu sterowania, na przykład:

```
type Direction = 'up' | 'down';
const move = (direction: Direction): void => {
    switch (direction) {
        case 'up':
            // move up
            break;
        case 'down':
            // move down
            break;
        default:
            const exhaustiveCheck: never = direction;
                        throw new Error(`Unhandled direction:
${exhaustiveCheck}`);
    }
};
```

# Interfejs i typ

## Podstawowa składnia

W TypeScript interfejsy definiują strukturę obiektów, określając nazwy i typy właściwości lub metod, które musi mieć dany

obiekt. Podstawowa składnia definiowania interfejsu w TypeScript wygląda następująco:

```
interface InterfaceName {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
}
```

Podobnie wygląda definicja typu:

```
type TypeName = {
    property1: Type1;
    // ...
    method1(arg1: ArgType1, arg2: ArgType2): ReturnType;
    // ...
};
```

`interface InterfaceName` lub `type TypeName`: definiuje nazwę interfejsu lub typu. `property1: Type1`: określa właściwości interfejsu wraz z odpowiadającymi im typami. Można zdefiniować wiele właściwości, oddzielając każdą z nich średnikiem. `method1(arg1: ArgType1, arg2: ArgType2): ReturnType;`: określa metody interfejsu. Metody definiuje się za pomocą ich nazw, po których następuje ujęta w nawiasy lista parametrów oraz typ zwracany. Można zdefiniować wiele metod, oddzielając każdą z nich średnikiem.

Przykład interfejsu:

```
interface Person {
    name: string;
    age: number;
    greet(): void;
}
```

Przykład typu:

```
type TypeName = {
    property1: string;
    method1(arg1: string, arg2: string): string;
};
```

W TypeScript typy służą do definiowania kształtu danych i wymuszania kontroli typów. Istnieje kilka często używanych składni definiowania typów w TypeScript, zależnie od konkretnego przypadku użycia. Oto kilka przykładów:

### Typy podstawowe

```
let myNumber: number = 123; // number type
let myBoolean: boolean = true; // boolean type
let myArray: string[] = ['a', 'b']; // array of strings
let myTuple: [string, number] = ['a', 123]; // tuple
```

### Obiekty i interfejsy

```
const x: { name: string; age: number } = { name: 'Simon', age:
7 };
```

### Typy sumy i przecięcia

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection type
let myCombined: CombinedType = { name: 'John', age: 25 }; //
Object with both name and age properties
```

# Wbudowane typy proste

TypeScript ma kilka wbudowanych typów prostych, których można używać do definiowania zmiennych, parametrów funkcji i typów zwracanych:

- `number`: reprezentuje wartości liczbowe, w tym liczby całkowite i zmiennoprzecinkowe.
- `string`: reprezentuje dane tekstowe.
- `boolean`: reprezentuje wartości logiczne, które mogą mieć wartość true albo false.
- `null`: reprezentuje brak wartości.
- `undefined`: reprezentuje wartość, która nie została przypisana ani zdefiniowana.
- `symbol`: reprezentuje unikatowy identyfikator. Symbole są zazwyczaj używane jako klucze właściwości obiektów.
- `bigint`: reprezentuje liczby całkowite o dowolnej precyzji.
- `any`: reprezentuje typ dynamiczny lub nieznany. Zmienne typu any mogą przechowywać wartości dowolnego typu i omijają kontrolę typów.
- `void`: reprezentuje brak jakiegokolwiek typu. Jest powszechnie używany jako typ zwracany funkcji, które nie zwracają wartości.
- `never`: reprezentuje typ wartości, które nigdy nie występują. Jest zazwyczaj używany jako typ zwracany funkcji, które zgłaszają błąd lub wchodzą w nieskończoną pętlę.

# Typowe wbudowane obiekty JavaScript

TypeScript jest nadzbiorem języka JavaScript, dlatego obejmuje wszystkie powszechnie używane wbudowane obiekty JavaScript. Pełną listę tych obiektów można znaleźć w dokumentacji Mozilla Developer Network (MDN):

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Oto lista niektórych powszechnie używanych wbudowanych obiektów JavaScript:

- Function
- Object
- Boolean
- Error
- Number
- BigInt
- Math
- Date
- String
- RegExp
- Array
- Map
- Set
- Promise
- Intl

# Przeciążenia

Przeciążenia funkcji w TypeScript pozwalają zdefiniować wiele sygnatur funkcji dla jednej nazwy, umożliwiając tworzenie funkcji, które można wywoływać na wiele sposobów. Oto przykład:

```
// Overloads
function sayHi(name: string): string;
function sayHi(names: string[]): string[];
```

```
// Implementation
function sayHi(name: unknown): unknown {
    if (typeof name === 'string') {
        return `Hi, ${name}!`;
    } else if (Array.isArray(name)) {
        return name.map(name => `Hi, ${name}!`);
    }
    throw new Error('Invalid value');
}

sayHi('xx'); // Valid
sayHi(['aa', 'bb']); // Valid
```

Oto kolejny przykład użycia przeciążeń funkcji wewnątrz deklaracji `class`:

```
class Greeter {
    message: string;

    constructor(message: string) {
        this.message = message;
    }

    // overload
    sayHi(name: string): string;
    sayHi(names: string[]): ReadonlyArray<string>;

    // implementation
    sayHi(name: unknown): unknown {
        if (typeof name === 'string') {
            return `${this.message}, ${name}!`;
        } else if (Array.isArray(name)) {
                    return  name.map(name  =>  `${this.message},
${name}!`);
        }
        throw new Error('value is invalid');
    }
```

```
}
console.log(new Greeter('Hello').sayHi('Simon'));
```

## Scalanie i rozszerzanie

Scalanie i rozszerzanie odnoszą się do dwóch różnych koncepcji związanych z pracą z typami i interfejsami.

Scalanie pozwala połączyć wiele deklaracji o tej samej nazwie w jedną definicję, na przykład gdy wielokrotnie definiuje się interfejs o tej samej nazwie:

```
interface X {
    a: string;
}

interface X {
    b: number;
}

const person: X = {
    a: 'a',
    b: 7,
};
```

Rozszerzanie oznacza możliwość rozszerzania istniejących typów lub interfejsów albo dziedziczenia po nich w celu tworzenia nowych. Jest to mechanizm dodawania właściwości lub metod do istniejącego typu bez modyfikowania jego pierwotnej definicji. Przykład:

```
interface Animal {
    name: string;
    eat(): void;
}
```

```
interface Bird extends Animal {
    sing(): void;
}

const dog: Bird = {
    name: 'Bird 1',
    eat() {
        console.log('Eating');
    },
    sing() {
        console.log('Singing');
    },
};
```

# Różnice między typem a interfejsem

Scalanie deklaracji (rozszerzanie):

Interfejsy obsługują scalanie deklaracji, co oznacza, że można zdefiniować wiele interfejsów o tej samej nazwie, a TypeScript scali je w jeden interfejs zawierający połączone właściwości i metody. Typy natomiast nie obsługują scalania deklaracji. Może to być przydatne, gdy chce się dodać funkcjonalność lub dostosować istniejące typy bez modyfikowania pierwotnych definicji albo poprawić brakujące lub nieprawidłowe typy.

```
interface A {
    x: string;
}
interface A {
    y: string;
}
const j: A = {
    x: 'xx',
    y: 'yy',
};
```

Rozszerzanie innych typów/interfejsów:

Zarówno typy, jak i interfejsy mogą rozszerzać inne typy/interfejsy, ale składnia jest inna. W przypadku interfejsów słowo kluczowe `extends` służy do dziedziczenia właściwości i metod z innych interfejsów. Interfejs nie może jednak rozszerzać typu złożonego, takiego jak typ sumy.

```
interface A {
    x: string;
    y: number;
}
interface B extends A {
    z: string;
}
const car: B = {
    x: 'x',
    y: 123,
    z: 'z',
};
```

W przypadku typów operator `&` służy do łączenia wielu typów w jeden typ (przecięcie).

```
interface A {
    x: string;
    y: number;
}

type B = A & {
    j: string;
};

const c: B = {
    x: 'x',
    y: 123,
```

```
    j: 'j',
};
```

Typy sumy i przecięcia:

Typy są bardziej elastyczne podczas definiowania typów sumy i przecięcia. Za pomocą słowa kluczowego `type` można łatwo tworzyć typy sumy przy użyciu operatora `|` oraz typy przecięcia przy użyciu operatora `&`. Interfejsy mogą wprawdzie pośrednio reprezentować typy sumy, ale nie mają wbudowanej obsługi typów przecięcia.

```
type Department = 'dep-x' | 'dep-y'; // Union

type Person = {
    name: string;
    age: number;
};

type Employee = {
    id: number;
    department: Department;
};

type EmployeeInfo = Person & Employee; // Intersection
```

Przykład z interfejsami:

```
interface A {
    x: 'x';
}
interface B {
    y: 'y';
}

type C = A | B; // Union of interfaces
```

# Klasa

## Podstawowa składnia klasy

Słowo kluczowe `class` służy w TypeScript do definiowania klasy. Poniżej znajduje się przykład:

```
class Person {
    private name: string;
    private age: number;
    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }
    public sayHi(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
```

Słowo kluczowe `class` służy do zdefiniowania klasy o nazwie „Person”.

Klasa ma dwie prywatne właściwości: name typu `string` i age typu `number`.

Konstruktor jest definiowany za pomocą słowa kluczowego `constructor`. Przyjmuje name i age jako parametry i przypisuje je do odpowiadających im właściwości.

Klasa ma metodę `public` o nazwie sayHi, która zapisuje w konsoli wiadomość powitalną.

Aby utworzyć instancję klasy w TypeScript, można użyć słowa kluczowego `new`, po którym następuje nazwa klasy i nawiasy `()`. Na przykład:

```
const myObject = new Person('John Doe', 25);
myObject.sayHi(); // Output: Hello, my name is John Doe and I
am 25 years old.
```

## Konstruktor

Konstruktory to specjalne metody w klasie, które służą do inicjalizowania właściwości obiektu podczas tworzenia instancji klasy.

```
class Person {
    public name: string;
    public age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    sayHello() {
        console.log(
            `Hello, my name is ${this.name} and I'm ${this.age}
years old.`
        );
    }
}

const john = new Person('Simon', 17);
john.sayHello();
```

Konstruktor można przeciążyć przy użyciu następującej składni:

```ts
type Sex = 'm' | 'f';

class Person {
    name: string;
    age: number;
    sex: Sex;

    constructor(name: string, age: number, sex?: Sex);
    constructor(name: string, age: number, sex: Sex) {
        this.name = name;
        this.age = age;
        this.sex = sex ?? 'm';
    }
}

const p1 = new Person('Simon', 17);
const p2 = new Person('Alice', 22, 'f');
```

W TypeScript można zdefiniować wiele przeciążeń konstruktora, ale tylko jedną implementację, która musi być zgodna ze wszystkimi przeciążeniami. Można to osiągnąć za pomocą parametru opcjonalnego.

```ts
class Person {
    name: string;
    age: number;

    constructor();
    constructor(name: string);
    constructor(name: string, age: number);
    constructor(name?: string, age?: number) {
        this.name = name ?? 'Unknown';
        this.age = age ?? 0;
    }

    displayInfo() {
        console.log(`Name: ${this.name}, Age: ${this.age}`);
    }
```

```
}

const person1 = new Person();
person1.displayInfo(); // Name: Unknown, Age: 0

const person2 = new Person('John');
person2.displayInfo(); // Name: John, Age: 0

const person3 = new Person('Jane', 25);
person3.displayInfo(); // Name: Jane, Age: 25
```

## Konstruktory prywatne i chronione

W TypeScript konstruktory mogą być oznaczone jako prywatne lub chronione, co ogranicza ich dostępność i użycie.

Konstruktory prywatne: Mogą być wywoływane wyłącznie wewnątrz samej klasy. Konstruktory prywatne są często używane w sytuacjach, gdy chce się wymusić wzorzec singletonu lub ograniczyć tworzenie instancji do metody wytwórczej wewnątrz klasy.

Konstruktory chronione: Konstruktory chronione są przydatne, gdy chce się utworzyć klasę bazową, której nie należy bezpośrednio instancjonować, ale którą mogą rozszerzać klasy pochodne.

```
class BaseClass {
    protected constructor() {}
}

class DerivedClass extends BaseClass {
    private value: number;

    constructor(value: number) {
        super();
```

```
        this.value = value;
    }
}

// Attempting to instantiate the base class directly will
result in an error
// const baseObj = new BaseClass(); // Error: Constructor of
class 'BaseClass' is protected.

// Create an instance of the derived class
const derivedObj = new DerivedClass(10);
```

## Modyfikatory dostępu

Modyfikatory dostępu `private`, `protected` i `public` służą do kontrolowania widoczności i dostępności elementów klas TypeScript, takich jak właściwości i metody. Modyfikatory te są niezbędne do wymuszania hermetyzacji oraz wyznaczania granic dostępu do wewnętrznego stanu klasy i jego modyfikacji.

Modyfikator `private` ogranicza dostęp do elementu klasy wyłącznie do klasy, w której się on znajduje.

Modyfikator `protected` umożliwia dostęp do elementu klasy wewnątrz klasy, w której się on znajduje oraz w jej klasach pochodnych.

Modyfikator `public` zapewnia nieograniczony dostęp do elementu klasy, umożliwiając dostęp do niego z dowolnego miejsca.

## Gettery i settery

Gettery i settery to specjalne metody umożliwiające definiowanie niestandardowego sposobu dostępu do

właściwości klasy i ich modyfikowania. Pozwalają hermetyzować wewnętrzny stan obiektu oraz dodawać logikę podczas pobierania lub ustawiania wartości właściwości. W TypeScript gettery i settery definiuje się odpowiednio za pomocą słów kluczowych `get` i `set`. Oto przykład:

```
class MyClass {
    private _myProperty: string;

    constructor(value: string) {
        this._myProperty = value;
    }
    get myProperty(): string {
        return this._myProperty;
    }
    set myProperty(value: string) {
        this._myProperty = value;
    }
}
```

## Automatyczne akcesory w klasach

TypeScript w wersji 4.9 dodaje obsługę automatycznych akcesorów, nadchodzącej funkcji ECMAScript. Przypominają właściwości klas, ale są deklarowane za pomocą słowa kluczowego `accessor`.

```
class Animal {
    accessor name: string;

    constructor(name: string) {
        this.name = name;
    }
}
```

Automatyczne akcesory są przekształcane w prywatne akcesory `get` i `set`, które operują na niedostępnej właściwości.

```
class Animal {
    #__name: string;

    get name() {
        return this.#__name;
    }
    set name(value: string) {
        this.#__name = value;
    }

    constructor(name: string) {
        this.name = name;
    }
}
```

## this

W TypeScript słowo kluczowe `this` odnosi się do bieżącej instancji klasy wewnątrz jej metod lub konstruktorów. Umożliwia dostęp do właściwości i metod klasy oraz ich modyfikowanie z poziomu jej własnego zakresu. Zapewnia sposób uzyskiwania dostępu do wewnętrznego stanu obiektu i manipulowania nim wewnątrz jego własnych metod.

```
class Person {
    private name: string;
    constructor(name: string) {
        this.name = name;
    }
    public introduce(): void {
        console.log(`Hello, my name is ${this.name}.`);
    }
}
```

```
const person1 = new Person('Alice');
person1.introduce(); // Hello, my name is Alice.
```

## Właściwości parametrów

Właściwości parametrów pozwalają deklarować i inicjalizować właściwości klasy bezpośrednio w parametrach konstruktora, eliminując powtarzalny kod. Na przykład:

```
class Person {
    constructor(
        private name: string,
        public age: number
    ) {
            // The "private" and "public" keywords in the
constructor
              // automatically declare and initialize the
corresponding class properties.
    }
    public introduce(): void {
        console.log(
                    `Hello, my name is ${this.name} and I am
${this.age} years old.`
        );
    }
}
const person = new Person('Alice', 25);
person.introduce();
```

## Klasy abstrakcyjne

Klasy abstrakcyjne są używane w TypeScript głównie do dziedziczenia. Umożliwiają definiowanie wspólnych właściwości i metod, które mogą być dziedziczone przez klasy pochodne. Jest to przydatne, gdy chce się zdefiniować wspólne zachowanie i wymusić implementację określonych metod przez klasy pochodne. Klasy abstrakcyjne pozwalają tworzyć

hierarchię klas, w której abstrakcyjna klasa bazowa zapewnia wspólny interfejs i wspólną funkcjonalność dla klas pochodnych.

```
abstract class Animal {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }

    abstract makeSound(): void;
}

class Cat extends Animal {
    makeSound(): void {
        console.log(`${this.name} meows.`);
    }
}

const cat = new Cat('Whiskers');
cat.makeSound(); // Output: Whiskers meows.
```

## Z typami generycznymi

Klasy z typami generycznymi umożliwiają definiowanie klas wielokrotnego użytku, które mogą działać z różnymi typami.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
```

```
    }

    setItem(item: T): void {
        this.item = item;
    }
}

const container1 = new Container<number>(42);
console.log(container1.getItem()); //  42

const container2 = new Container<string>('Hello');
container2.setItem('World');
console.log(container2.getItem()); // World
```

## Dekoratory

Dekoratory zapewniają mechanizm dodawania metadanych, modyfikowania zachowania, walidowania lub rozszerzania funkcjonalności elementu docelowego. Są to funkcje wykonywane w czasie działania programu. Do jednej deklaracji można zastosować wiele dekoratorów.

Dekoratory są eksperymentalną funkcjonalnością, a poniższe przykłady są zgodne wyłącznie z TypeScript w wersji 5 lub nowszej przy użyciu ES6.

W wersjach TypeScript starszych niż 5 należy je włączyć za pomocą właściwości `experimentalDecorators` w pliku `tsconfig.json` lub opcji `--experimentalDecorators` w wierszu poleceń (jednak poniższy przykład nie zadziała).

Niektóre z typowych przypadków użycia dekoratorów obejmują:

- Obserwowanie zmian właściwości.

- Obserwowanie wywołań metod.
- Dodawanie dodatkowych właściwości lub metod.
- Walidację w czasie działania programu.
- Automatyczną serializację i deserializację.
- Rejestrowanie zdarzeń.
- Autoryzację i uwierzytelnianie.
- Ochronę przed błędami.

Uwaga: dekoratory w wersji 5 nie pozwalają dekorować parametrów.

Typy dekoratorów:

### Dekoratory klas

Dekoratory klas są przydatne do rozszerzania istniejącej klasy, na przykład przez dodawanie właściwości lub metod albo gromadzenie instancji klasy. W poniższym przykładzie dodajemy metodę `toString`, która przekształca klasę w reprezentację tekstową.

```
type Constructor<T = {}> = new (...args: any[]) => T;

function toString<Class extends Constructor>(
    Value: Class,
    context: ClassDecoratorContext<Class>
) {
    return class extends Value {
        constructor(...args: any[]) {
            super(...args);
            console.log(JSON.stringify(this));
            console.log(JSON.stringify(context));
        }
    };
}
```

```
@toString
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    greet() {
        return 'Hello, ' + this.name;
    }
}
const person = new Person('Simon');
/* Logs:
{"name":"Simon"}
{"kind":"class","name":"Person"}
*/
```

### Dekorator właściwości

Dekoratory właściwości są przydatne do modyfikowania zachowania właściwości, na przykład przez zmianę wartości początkowych. W poniższym kodzie znajduje się skrypt, który sprawia, że właściwość jest zawsze zapisywana wielkimi literami:

```
function upperCase<T>(
    target: undefined,
    context: ClassFieldDecoratorContext<T, string>
) {
    return function (this: T, value: string) {
        return value.toUpperCase();
    };
}

class MyClass {
```

```
    @upperCase
    prop1 = 'hello!';
}

console.log(new MyClass().prop1); // Logs: HELLO!
```

### Dekorator metody

Dekoratory metod umożliwiają zmianę lub rozszerzenie zachowania metod. Poniżej znajduje się przykład prostego rejestratora:

```
function log<This, Args extends any[], Return>(
    target: (this: This, ...args: Args) => Return,
    context: ClassMethodDecoratorContext<
        This,
        (this: This, ...args: Args) => Return
    >
) {
    const methodName = String(context.name);

       function replacementMethod(this: This, ...args: Args):
Return {
        console.log(`LOG: Entering method '${methodName}'.`);
        const result = target.call(this, ...args);
        console.log(`LOG: Exiting method '${methodName}'.`);
        return result;
    }

    return replacementMethod;
}

class MyClass {
    @log
    sayHello() {
        console.log('Hello!');
    }
```

```
}

new MyClass().sayHello();
```

Rejestrowane są następujące komunikaty:

```
LOG: Entering method 'sayHello'.
Hello!
LOG: Exiting method 'sayHello'.
```

### Dekoratory getterów i setterów

Dekoratory getterów i setterów umożliwiają zmianę lub rozszerzenie zachowania akcesorów klasy. Są przydatne na przykład do walidowania przypisań właściwości. Oto prosty przykład dekoratora gettera:

```
function range<This, Return extends number>(min: number, max:
number) {
    return function (
        target: (this: This) => Return,
        context: ClassGetterDecoratorContext<This, Return>
    ) {
        return function (this: This): Return {
            const value = target.call(this);
            if (value < min || value > max) {
                throw 'Invalid';
            }
            Object.defineProperty(this, context.name, {
                value,
                enumerable: true,
            });
            return value;
        };
    };
}
```

```
class MyClass {
    private _value = 0;

    constructor(value: number) {
        this._value = value;
    }
    @range(1, 100)
    get getValue(): number {
        return this._value;
    }
}

const obj = new MyClass(10);
console.log(obj.getValue); // Valid: 10

const obj2 = new MyClass(999);
console.log(obj2.getValue); // Throw: Invalid!
```

### Metadane dekoratorów

Metadane dekoratorów upraszczają proces stosowania i wykorzystywania metadanych przez dekoratory w dowolnej klasie. Dekoratory mogą uzyskać dostęp do nowej właściwości `metadata` w obiekcie kontekstu, która może służyć jako klucz zarówno dla wartości prostych, jak i obiektów. Dostęp do informacji o metadanych można uzyskać w klasie za pomocą `Symbol.metadata`.

Metadanych można używać do różnych celów, takich jak debugowanie, serializacja lub wstrzykiwanie zależności przy użyciu dekoratorów.

```
//@ts-ignore
Symbol.metadata ??= Symbol('Symbol.metadata'); // Simple
polyfill
```

```
type Context =
    | ClassFieldDecoratorContext
    | ClassAccessorDecoratorContext
    | ClassMethodDecoratorContext; // Context contains property
metadata: DecoratorMetadata

function setMetadata(_target: any, context: Context) {
    // Set the metadata object with a primitive value
    context.metadata[context.name] = true;
}

class MyClass {
    @setMetadata
    a = 123;

    @setMetadata
    accessor b = 'b';

    @setMetadata
    fn() {}
}

const metadata = MyClass[Symbol.metadata]; // Get metadata
information

console.log(JSON.stringify(metadata));                    //
{"bar":true,"baz":true,"foo":true}
```

## Dziedziczenie

Dziedziczenie odnosi się do mechanizmu, dzięki któremu klasa może dziedziczyć właściwości i metody innej klasy, nazywanej klasą bazową lub nadrzędną. Klasa pochodna, nazywana również klasą potomną lub podklasą, może rozszerzać i specjalizować funkcjonalność klasy bazowej przez dodawanie nowych właściwości i metod albo nadpisywanie już istniejących.

```
class Animal {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    speak(): void {
        console.log('The animal makes a sound');
    }
}

class Dog extends Animal {
    breed: string;

    constructor(name: string, breed: string) {
        super(name);
        this.breed = breed;
    }

    speak(): void {
        console.log('Woof! Woof!');
    }
}

// Create an instance of the base class
const animal = new Animal('Generic Animal');
animal.speak(); // The animal makes a sound

// Create an instance of the derived class
const dog = new Dog('Max', 'Labrador');
dog.speak(); // Woof! Woof!"
```

TypeScript nie obsługuje dziedziczenia wielokrotnego w tradycyjnym znaczeniu, lecz pozwala na dziedziczenie z jednej klasy bazowej. TypeScript umożliwia implementowanie wielu interfejsów. Interfejs może definiować kontrakt dotyczący

struktury obiektu, a klasa może implementować wiele interfejsów. Dzięki temu klasa może dziedziczyć zachowanie i strukturę z wielu źródeł.

```
interface Flyable {
    fly(): void;
}

interface Swimmable {
    swim(): void;
}

class FlyingFish implements Flyable, Swimmable {
    fly() {
        console.log('Flying...');
    }

    swim() {
        console.log('Swimming...');
    }
}

const flyingFish = new FlyingFish();
flyingFish.fly();
flyingFish.swim();
```

Słowo kluczowe `class` w TypeScript, podobnie jak w JavaScript, jest często określane mianem lukru składniowego. Zostało wprowadzone w ECMAScript 2015 (ES6), aby zapewnić bardziej znajomą składnię tworzenia obiektów i pracy z nimi w sposób oparty na klasach. Należy jednak pamiętać, że TypeScript, jako nadzbiór JavaScript, jest ostatecznie kompilowany do JavaScript, który w swojej istocie pozostaje oparty na prototypach.

## Elementy statyczne

TypeScript obsługuje elementy statyczne. Aby uzyskać dostęp do statycznych elementów klasy, można użyć nazwy klasy, po której następuje kropka, bez potrzeby tworzenia obiektu.

```
class OfficeWorker {
    static memberCount: number = 0;

    constructor(private name: string) {
        OfficeWorker.memberCount++;
    }
}

const w1 = new OfficeWorker('James');
const w2 = new OfficeWorker('Simon');
const total = OfficeWorker.memberCount;
console.log(total); // 2
```

## Inicjalizacja właściwości

Istnieje kilka sposobów inicjalizowania właściwości klasy w TypeScript:

Bezpośrednio:

W poniższym przykładzie te wartości początkowe zostaną użyte podczas tworzenia instancji klasy.

```
class MyClass {
    property1: string = 'default value';
    property2: number = 42;
}
```

W konstruktorze:

```
class MyClass {
    property1: string;
    property2: number;
```

```
    constructor() {
        this.property1 = 'default value';
        this.property2 = 42;
    }
}
```

Za pomocą parametrów konstruktora:

```
class MyClass {
    constructor(
        private property1: string = 'default value',
        public property2: number = 42
    ) {
            // There is no need to assign the values to the
properties explicitly.
    }
    log() {
        console.log(this.property2);
    }
}
const x = new MyClass();
x.log();
```

## Przeciążanie metod

Przeciążanie metod pozwala klasie mieć wiele metod o tej samej nazwie, ale z różnymi typami parametrów lub różną liczbą parametrów. Dzięki temu można wywoływać metodę na różne sposoby, zależnie od przekazanych argumentów.

```
class MyClass {
    add(a: number, b: number): number; // Overload signature 1
    add(a: string, b: string): string; // Overload signature 2

      add(a: number | string, b: number | string): number |
string {
        if (typeof a === 'number' && typeof b === 'number') {
```

```
            return a + b;
        }
        if (typeof a === 'string' && typeof b === 'string') {
            return a.concat(b);
        }
        throw new Error('Invalid arguments');
    }
}

const r = new MyClass();
console.log(r.add(10, 5)); // Logs 15
```

# Typy generyczne

Typy generyczne umożliwiają tworzenie komponentów i funkcji wielokrotnego użytku, które mogą współpracować z wieloma typami. Dzięki typom generycznym można parametryzować typy, funkcje i interfejsy, co pozwala im działać na różnych typach bez konieczności ich wcześniejszego jawnego określania.

Typy generyczne pozwalają tworzyć bardziej elastyczny kod wielokrotnego użytku.

## Typ generyczny

Aby zdefiniować typ generyczny, należy użyć nawiasów ostrych (<>) do określenia parametrów typu, na przykład:

```
function identity<T>(arg: T): T {
    return arg;
}
const a = identity('x');
const b = identity(123);
```

```
const getLen = <T,>(data: ReadonlyArray<T>) => data.length;
const len = getLen([1, 2, 3]);
```

## Klasy generyczne

Typy generyczne można również stosować do klas, dzięki czemu mogą one współpracować z wieloma typami za pomocą parametrów typu. Jest to przydatne podczas tworzenia definicji klas wielokrotnego użytku, które mogą działać na różnych typach danych przy zachowaniu bezpieczeństwa typów.

```
class Container<T> {
    private item: T;

    constructor(item: T) {
        this.item = item;
    }

    getItem(): T {
        return this.item;
    }
}

const numberContainer = new Container<number>(123);
console.log(numberContainer.getItem()); // 123

const stringContainer = new Container<string>('hello');
console.log(stringContainer.getItem()); // hello
```

## Ograniczenia typów generycznych

Parametry generyczne można ograniczać za pomocą słowa kluczowego `extends`, po którym podaje się typ lub interfejs, z którym parametr typu musi być zgodny.

W poniższym przykładzie typ `T` musi mieć prawidłowo typowaną właściwość `length`, aby był poprawny:

```
const printLen = <T extends { length: number }>(value: T): void
=> {
    console.log(value.length);
};

printLen('Hello'); // 5
printLen([1, 2, 3]); // 3
printLen({ length: 10 }); // 10
printLen(123); // Invalid
```

Istotną funkcją typów generycznych, wprowadzoną w wersji 3.4 RC, jest wnioskowanie typów dla funkcji wyższego rzędu, które propaguje argumenty typów generycznych:

```
declare function pipe<A extends any[], B, C>(
    ab: (...args: A) => B,
    bc: (b: B) => C
): (...args: A) => C;

declare function list<T>(a: T): T[];
declare function box<V>(x: V): { value: V };

const listBox = pipe(list, box); // <T>(a: T) => { value: T[] }
const boxList = pipe(box, list); // <V>(x: V) => { value: V }[]
```

Ta funkcjonalność ułatwia bezpieczne pod względem typów programowanie w stylu bezpunktowym (point-free), często spotykanym w programowaniu funkcyjnym.

## Kontekstowe zawężanie typów generycznych

Kontekstowe zawężanie typów generycznych to mechanizm TypeScriptu, który umożliwia kompilatorowi zawężenie typu

parametru generycznego na podstawie kontekstu, w którym jest on używany. Jest to przydatne podczas pracy z typami generycznymi w instrukcjach warunkowych:

```
function process<T>(value: T): void {
    if (typeof value === 'string') {
        // Value is narrowed down to type 'string'
        console.log(value.length);
    } else if (typeof value === 'number') {
        // Value is narrowed down to type 'number'
        console.log(value.toFixed(2));
    }
}

process('hello'); // 5
process(3.14159); // 3.14
```

# Wymazywane typy strukturalne

W TypeScripcie obiekty nie muszą dokładnie odpowiadać konkretnemu typowi. Jeśli na przykład utworzymy obiekt spełniający wymagania interfejsu, możemy użyć go w miejscach, w których wymagany jest ten interfejs, nawet jeśli nie istnieje między nimi jawne powiązanie. Przykład:

```
type NameProp1 = {
    prop1: string;
};

function log(x: NameProp1) {
    console.log(x.prop1);
}

const obj = {
    prop2: 123,
    prop1: 'Origin',
```

```
};

log(obj); // Valid
```

# Przestrzenie nazw

W TypeScripcie przestrzenie nazw służą do organizowania kodu w logiczne kontenery, co zapobiega konfliktom nazw i umożliwia grupowanie powiązanego kodu. Użycie słowa kluczowego `export` umożliwia dostęp do przestrzeni nazw spoza modułów.

```
export namespace MyNamespace {
    export interface MyInterface1 {
        prop1: boolean;
    }
    export interface MyInterface2 {
        prop2: string;
    }
}

const a: MyNamespace.MyInterface1 = {
    prop1: true,
};
```

# Symbole

Symbole są prymitywnym typem danych reprezentującym niezmienną wartość, która przez cały czas działania programu ma zagwarantowaną globalną unikatowość.

Symbole mogą być używane jako klucze właściwości obiektów i umożliwiają tworzenie właściwości niewyliczalnych.

```ts
const key1: symbol = Symbol('key1');
const key2: symbol = Symbol('key2');

const obj = {
    [key1]: 'value 1',
    [key2]: 'value 2',
};

console.log(obj[key1]); // value 1
console.log(obj[key2]); // value 2
```

Symbole mogą być teraz używane jako klucze w obiektach WeakMap i WeakSet.

## Dyrektywy z potrójnym ukośnikiem

Dyrektywy z potrójnym ukośnikiem to specjalne komentarze, które przekazują kompilatorowi instrukcje dotyczące sposobu przetwarzania pliku. Dyrektywy te zaczynają się od trzech kolejnych ukośników (`///`), są zazwyczaj umieszczane na początku pliku TypeScript i nie mają wpływu na zachowanie programu w czasie wykonywania.

Dyrektywy z potrójnym ukośnikiem służą między innymi do odwoływania się do zewnętrznych zależności, określania sposobu ładowania modułów oraz włączania lub wyłączania określonych funkcji kompilatora. Oto kilka przykładów:

Odwołanie do pliku deklaracji:

```ts
/// <reference path="path/to/declaration/file.d.ts" />
```

Wskazanie formatu modułu:

```ts
/// <amd|commonjs|system|umd|es6|es2015|none>
```

Włączenie opcji kompilatora — w poniższym przykładzie trybu ścisłego:

```
/// <strict|noImplicitAny|noUnusedLocals|noUnusedParameters>
```

# Manipulowanie typami

## Tworzenie typów na podstawie innych typów

Można tworzyć nowe typy przez komponowanie, modyfikowanie lub przekształcanie istniejących typów.

Typy przecięcia (`&`):

Umożliwiają połączenie wielu typów w jeden typ:

```
type A = { foo: number };
type B = { bar: string };
type C = A & B; // Intersection of A and B
const obj: C = { foo: 42, bar: 'hello' };
```

Typy sumy (`|`):

Umożliwiają zdefiniowanie typu, który może być jednym z kilku typów:

```
type Result = string | number;
const value1: Result = 'hello';
const value2: Result = 42;
```

Typy mapowane:

Umożliwiają przekształcenie właściwości istniejącego typu w celu utworzenia nowego typu:

```
type Mutable<T> = {
    readonly [P in keyof T]: T[P];
};
type Person = {
    name: string;
    age: number;
};
type ImmutablePerson = Mutable<Person>; // Properties become
read-only
```

Typy warunkowe:

Umożliwiają tworzenie typów na podstawie określonych warunków:

```
type ExtractParam<T> = T extends (param: infer P) => any ? P :
never;
type MyFunction = (name: string) => number;
type ParamType = ExtractParam<MyFunction>; // string
```

## Typy dostępu indeksowanego

W TypeScripcie można uzyskiwać dostęp do typów właściwości w innym typie i modyfikować je za pomocą indeksu `Type[Key]`.

```
type Person = {
    name: string;
    age: number;
};

type AgeType = Person['age']; // number

type MyTuple = [string, number, boolean];
type MyType = MyTuple[2]; // boolean
```

## Typy narzędziowe

Do manipulowania typami można używać kilku wbudowanych typów narzędziowych. Poniżej znajduje się lista najczęściej używanych:

### Awaited<T>

Tworzy typ, który rekurencyjnie rozpakowuje typy `Promise`.

```
type A = Awaited<Promise<string>>; // string
```

### Partial<T>

Tworzy typ, w którym wszystkie właściwości typu T są opcjonalne.

```
type Person = {
    name: string;
    age: number;
};

type A = Partial<Person>; // { name?: string | undefined; age?:
number | undefined; }
```

### Required<T>

Tworzy typ, w którym wszystkie właściwości typu T są wymagane.

```
type Person = {
    name?: string;
    age?: number;
};

type A = Required<Person>; // { name: string; age: number; }
```

### Readonly<T>

Tworzy typ, w którym wszystkie właściwości typu T są tylko do odczytu.

```
type Person = {
    name: string;
    age: number;
};

type A = Readonly<Person>;

const a: A = { name: 'Simon', age: 17 };
a.name = 'John'; // Invalid
```

### Record<K, T>

Tworzy typ z zestawem właściwości K typu T.

```
type Product = {
    name: string;
    price: number;
};

const products: Record<string, Product> = {
    apple: { name: 'Apple', price: 0.5 },
    banana: { name: 'Banana', price: 0.25 },
};

console.log(products.apple); // { name: 'Apple', price: 0.5 }
```

### Pick<T, K>

Tworzy typ przez wybranie określonych właściwości K z typu T.

```ts
type Product = {
    name: string;
    price: number;
};

type Price = Pick<Product, 'price'>; // { price: number; }
```

### Omit<T, K>

Tworzy typ przez pominięcie określonych właściwości K z typu T.

```ts
type Product = {
    name: string;
    price: number;
};

type Name = Omit<Product, 'price'>; // { name: string; }
```

### Exclude<T, U>

Tworzy typ przez wykluczenie z typu T wszystkich wartości typu U.

```ts
type Union = 'a' | 'b' | 'c';
type MyType = Exclude<Union, 'a' | 'c'>; // b
```

### Extract<T, U>

Tworzy typ przez wyodrębnienie z typu T wszystkich wartości typu U.

```ts
type Union = 'a' | 'b' | 'c';
type MyType = Extract<Union, 'a' | 'c'>; // a | c
```

### NonNullable<T>

Tworzy typ przez wykluczenie wartości null i undefined z typu T.

```
type Union = 'a' | null | undefined | 'b';
type MyType = NonNullable<Union>; // 'a' | 'b'
```

### Parameters<T>

Wyodrębnia typy parametrów z typu funkcji T.

```
type Func = (a: string, b: number) => void;
type MyType = Parameters<Func>; // [a: string, b: number]
```

### ConstructorParameters<T>

Wyodrębnia typy parametrów z typu funkcji konstruktora T.

```
class Person {
    constructor(
        public name: string,
        public age: number
    ) {}
}
type  PersonConstructorParams  =  ConstructorParameters<typeof
Person>; // [name: string, age: number]
const params: PersonConstructorParams = ['John', 30];
const person = new Person(...params);
console.log(person); // Person { name: 'John', age: 30 }
```

### ReturnType<T>

Wyodrębnia typ zwracany przez typ funkcji T.

```
type Func = (name: string) => number;
type MyType = ReturnType<Func>; // number
```

### InstanceType<T>

Wyodrębnia typ instancji z typu klasy T.

```
class Person {
    name: string;

    constructor(name: string) {
        this.name = name;
    }

    sayHello() {
        console.log(`Hello, my name is ${this.name}!`);
    }
}

type PersonInstance = InstanceType<typeof Person>;

const person: PersonInstance = new Person('John');

person.sayHello(); // Hello, my name is John!
```

### ThisParameterType<T>

Wyodrębnia typ parametru „this” z typu funkcji T.

```
interface Person {
    name: string;
    greet(this: Person): void;
}
type PersonThisType = ThisParameterType<Person['greet']>; // Person
```

### OmitThisParameter<T>

Usuwa parametr „this” z typu funkcji T.

```
function capitalize(this: String) {
                        return      this[0].toUpperCase()      +
this.substring(1).toLowerCase();
}

type CapitalizeType = OmitThisParameter<typeof capitalize>; //
() => string
```

### ThisType<T>

Służy jako znacznik kontekstowego typu `this`.

```
type Logger = {
    log: (error: string) => void;
};

let   helperFunctions:   {   [name:   string]:   Function   }   &
ThisType<Logger> = {
    hello: function () {
         this.log('some error'); // Valid as "log" is a part of
"this".
        this.update(); // Invalid
    },
};
```

### Uppercase<T>

Zmienia nazwę wejściowego typu T na pisaną wielkimi literami.

```
type MyType = Uppercase<'abc'>; // "ABC"
```

### Lowercase<T>

Zmienia nazwę wejściowego typu T na pisaną małymi literami.

```
type MyType = Lowercase<'ABC'>; // "abc"
```

### Capitalize<T>

Zmienia pierwszą literę nazwy wejściowego typu T na wielką.

```
type MyType = Capitalize<'abc'>; // "Abc"
```

### Uncapitalize<T>

Zmienia pierwszą literę nazwy wejściowego typu T na małą.

```
type MyType = Uncapitalize<'Abc'>; // "abc"
```

### NoInfer<T>

`NoInfer` to typ narzędziowy zaprojektowany w celu blokowania automatycznego wnioskowania typów w obrębie funkcji generycznej.

Przykład:

```
// Automatic inference of types within the scope of a generic
function.
function fn<T extends string>(x: T[], y: T) {
    return x.concat(y);
}
const r = fn(['a', 'b'], 'c'); // Type here is ("a" | "b" |
"c")[]
```

Z użyciem `NoInfer`:

```
// Example function that uses NoInfer to prevent type inference
function fn2<T extends string>(x: T[], y: NoInfer<T>) {
    return x.concat(y);
}

const r2 = fn2(['a', 'b'], 'c'); // Error: Type Argument of
type '"c"' is not assignable to parameter of type '"a" | "b"'.
```

# Inne

## Obsługa błędów i wyjątków

TypeScript pozwala przechwytywać i obsługiwać błędy za pomocą standardowych mechanizmów obsługi błędów języka JavaScript:

Bloki try-catch-finally:

```
try {
    // Code that might throw an error
} catch (error) {
    // Handle the error
} finally {
    // Code that always executes, finally is optional
}
```

Można także obsługiwać różne typy błędów:

```
try {
    // Code that might throw different types of errors
} catch (error) {
    if (error instanceof TypeError) {
        // Handle TypeError
    } else if (error instanceof RangeError) {
        // Handle RangeError
    } else {
        // Handle other errors
```

```
    }
}
```

Niestandardowe typy błędów:

Można określić bardziej szczegółowe błędy, rozszerzając klasę `Error`:

```
class CustomError extends Error {
    constructor(message: string) {
        super(message);
        this.name = 'CustomError';
    }
}

throw new CustomError('This is a custom error.');
```

## Klasy domieszkowe

Klasy domieszkowe pozwalają łączyć i komponować zachowania z wielu klas w jedną klasę. Umożliwiają ponowne wykorzystywanie i rozszerzanie funkcjonalności bez potrzeby tworzenia głębokich łańcuchów dziedziczenia.

```
abstract class Identifiable {
    name: string = '';
    logId() {
        console.log('id:', this.name);
    }
}
abstract class Selectable {
    selected: boolean = false;
    select() {
        this.selected = true;
        console.log('Select');
    }
    deselect() {
```

```
        this.selected = false;
        console.log('Deselect');
    }
}
class MyClass {
    constructor() {}
}

// Extend MyClass to include the behavior of Identifiable and
Selectable
interface MyClass extends Identifiable, Selectable {}

// Function to apply mixins to a class
function applyMixins(source: any, baseCtors: any[]) {
    baseCtors.forEach(baseCtor => {

Object.getOwnPropertyNames(baseCtor.prototype).forEach(name   =>
{
            let descriptor = Object.getOwnPropertyDescriptor(
                baseCtor.prototype,
                name
            );
            if (descriptor) {
                  Object.defineProperty(source.prototype, name,
descriptor);
            }
        });
    });
}

// Apply the mixins to MyClass
applyMixins(MyClass, [Identifiable, Selectable]);
let o = new MyClass();
o.name = 'abc';
o.logId();
o.select();
```

## Asynchroniczne funkcje języka

Ponieważ TypeScript jest nadzbiorem języka JavaScript, ma wbudowane asynchroniczne funkcje języka JavaScript, takie jak:

Obietnice:

Obietnice są sposobem obsługi operacji asynchronicznych i ich wyników za pomocą metod takich jak `.then()` i `.catch()`, które pozwalają obsługiwać pomyślne zakończenie oraz błędy.

Więcej informacji: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

Async/await:

Słowa kluczowe `async/await` zapewniają składnię przypominającą kod synchroniczny podczas pracy z obietnicami. Słowo kluczowe `async` służy do definiowania funkcji asynchronicznej, a słowo kluczowe `await` jest używane wewnątrz funkcji asynchronicznej do wstrzymania wykonywania do czasu rozwiązania lub odrzucenia obietnicy.

Więcej informacji: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await

Następujące interfejsy API są dobrze obsługiwane w TypeScript:

Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

Web Workers: https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API

Shared Workers: https://developer.mozilla.org/en-US/docs/Web/API/SharedWorker

WebSocket: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

## Iteratory i generatory

Zarówno iteratory, jak i generatory są dobrze obsługiwane w TypeScript.

Iteratory to obiekty implementujące protokół iteratora, który umożliwia dostęp po kolei do elementów kolekcji lub sekwencji. Jest to struktura zawierająca wskaźnik do następnego elementu iteracji. Iteratory mają metodę `next()`, która zwraca następną wartość w sekwencji wraz z wartością logiczną właściwości `done`, wskazującą, czy sekwencja została zakończona.

```
class NumberIterator implements Iterable<number> {
    private current: number;

    constructor(
        private start: number,
        private end: number
    ) {
        this.current = start;
    }

    public next(): IteratorResult<number> {
        if (this.current <= this.end) {
            const value = this.current;
            this.current++;
            return { value, done: false };
        } else {
            return { value: undefined, done: true };
        }
```

```
    }

    [Symbol.iterator](): Iterator<number> {
        return this;
    }
}

const iterator = new NumberIterator(1, 3);

for (const num of iterator) {
    console.log(num);
}
```

Generatory to specjalne funkcje definiowane przy użyciu składni `function*`, która upraszcza tworzenie iteratorów. Używają słowa kluczowego `yield` do definiowania sekwencji wartości oraz automatycznie wstrzymują i wznawiają wykonywanie, gdy wymagane są wartości.

Generatory ułatwiają tworzenie iteratorów i są szczególnie przydatne podczas pracy z dużymi lub nieskończonymi sekwencjami.

Przykład:

```
function* numberGenerator(start: number, end: number):
Generator<number> {
    for (let i = start; i <= end; i++) {
        yield i;
    }
}

const generator = numberGenerator(1, 5);

for (const num of generator) {
    console.log(num);
}
```

TypeScript obsługuje również iteratory asynchroniczne i generatory asynchroniczne.

Więcej informacji:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator

## Dokumentacja TsDocs JSDoc

Podczas pracy z bazą kodu JavaScript można pomóc TypeScriptowi w wywnioskowaniu właściwego typu, używając komentarzy JSDoc z dodatkowymi adnotacjami dostarczającymi informacje o typie.

Przykład:

```
/**
 * Computes the power of a given number
 * @constructor
 * @param {number} base - The base value of the expression
  * @param {number} exponent - The exponent value of the expression
 */
function power(base: number, exponent: number) {
    return Math.pow(base, exponent);
}
power(10, 2); // function power(base: number, exponent: number): number
```

Pełna dokumentacja jest dostępna pod tym adresem: https://www.typescriptlang.org/docs/handbook/jsdoc-supported-types.html

Od wersji 3.7 można generować definicje typów `.d.ts` ze składni JavaScript JSDoc. Więcej informacji można znaleźć tutaj: https://www.typescriptlang.org/docs/handbook/declaration-files/dts-from-js.html

## @types

Pakiety należące do organizacji `@types` korzystają ze specjalnej konwencji nazewnictwa, która służy do dostarczania definicji typów dla istniejących bibliotek lub modułów JavaScript. Na przykład użycie:

```
npm install --save-dev @types/lodash
```

zainstaluje definicje typów biblioteki `lodash` w bieżącym projekcie.

Aby współtworzyć definicje typów pakietu `@types`, prześlij pull request do https://github.com/DefinitelyTyped/DefinitelyTyped.

## JSX

JSX (JavaScript XML) to rozszerzenie składni języka JavaScript, które pozwala pisać kod podobny do HTML w plikach JavaScript lub TypeScript. Jest powszechnie używane w React do definiowania struktury HTML.

TypeScript rozszerza możliwości JSX, zapewniając sprawdzanie typów i analizę statyczną.

Aby używać JSX, należy ustawić opcję kompilatora `jsx` w pliku `tsconfig.json`. Dwie typowe opcje konfiguracji:

- `preserve`: generuje pliki `.jsx` z niezmienionym JSX. Ta opcja nakazuje TypeScriptowi zachować składnię JSX bez zmian i nie przekształcać jej podczas procesu kompilacji. Można jej użyć, jeśli osobne narzędzie, takie jak Babel, obsługuje przekształcanie.
- `react`: włącza wbudowane przekształcanie JSX w TypeScript. Zostanie użyte `React.createElement`.

Wszystkie opcje są dostępne tutaj: https://www.typescriptlang.org/tsconfig#jsx

## Moduły ES6

TypeScript obsługuje ES6 (ECMAScript 2015) i wiele późniejszych wersji. Oznacza to, że można używać składni ES6, takiej jak funkcje strzałkowe, literały szablonowe, klasy, moduły, destrukturyzacja i inne.

Aby włączyć funkcje ES6 w projekcie, można określić właściwość `target` w pliku `tsconfig.json`.

Przykład konfiguracji:

```
{
  "compilerOptions": {
    "target": "es6",
    "module": "es6",
    "moduleResolution": "node",
    "sourceMap": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

## Operator potęgowania ES7

Operator potęgowania (`**`) oblicza wartość otrzymaną przez podniesienie pierwszego operandu do potęgi określonej przez drugi operand. Działa podobnie do `Math.pow()`, ale dodatkowo może przyjmować wartości BigInt jako operandy. TypeScript w pełni obsługuje ten operator po ustawieniu `target` w pliku `tsconfig.json` na `es2016` lub nowszy.

```
console.log(2 ** (2 ** 2)); // 16
```

## Instrukcja for-await-of

Jest to funkcja języka JavaScript w pełni obsługiwana w TypeScript, która pozwala iterować po asynchronicznych obiektach iterowalnych, gdy wersja docelowa to `es2018`.

```
async function* asyncNumbers(): AsyncIterableIterator<number> {
    yield Promise.resolve(1);
    yield Promise.resolve(2);
    yield Promise.resolve(3);
}

(async () => {
    for await (const num of asyncNumbers()) {
        console.log(num);
    }
})();
```

## Metawłaściwość new.target

W TypeScript można używać metawłaściwości `new.target`, która pozwala ustalić, czy funkcja lub konstruktor zostały wywołane przy użyciu operatora `new`. Umożliwia ona wykrycie, czy obiekt został utworzony w wyniku wywołania konstruktora.

```
class Parent {
    constructor() {
            console.log(new.target);  // Logs the constructor
function used to create an instance
    }
}

class Child extends Parent {
    constructor() {
        super();
    }
}

const parentX = new Parent(); // [Function: Parent]
const child = new Child(); // [Function: Child]
```

## Wyrażenia importu dynamicznego

Można warunkowo lub leniwie ładować moduły na żądanie za pomocą propozycji ECMAScript dotyczącej importu dynamicznego, która jest obsługiwana w TypeScript.

Składnia wyrażeń importu dynamicznego w TypeScript wygląda następująco:

```
async function renderWidget() {
    const container = document.getElementById('widget');
    if (container !== null) {
            const widget = await import('./widget'); // Dynamic
import
        widget.render(container);
    }
}

renderWidget();
```

## “tsc –watch”

To polecenie uruchamia kompilator TypeScript z parametrem `--watch`, umożliwiając automatyczną ponowną kompilację plików TypeScript za każdym razem, gdy zostaną zmodyfikowane.

```
tsc --watch
```

Począwszy od TypeScript w wersji 4.9, monitorowanie plików opiera się głównie na zdarzeniach systemu plików i automatycznie przełącza się na odpytywanie, jeśli nie można ustanowić obserwatora opartego na zdarzeniach.

## Operator asercji non-null

Operator asercji non-null (przyrostkowy !), nazywany również asercją określonego przypisania, jest funkcją TypeScript, która pozwala zadeklarować, że zmienna lub właściwość nie jest równa null ani undefined, nawet jeśli statyczna analiza typów w TypeScript sugeruje, że może tak być. Ta funkcja umożliwia usunięcie jawnego sprawdzania.

```
type Person = {
    name: string;
};

const printName = (person?: Person) => {
    console.log(`Name is ${person!.name}`);
};
```

## Deklaracje z wartościami domyślnymi

Deklaracje z wartościami domyślnymi są używane, gdy zmiennej lub parametrowi przypisano wartość domyślną. Oznacza to, że jeśli dla tej zmiennej lub tego parametru nie

zostanie podana żadna wartość, użyta zostanie wartość domyślna.

```
function greet(name: string = 'Anonymous'): void {
    console.log(`Hello, ${name}!`);
}
greet(); // Hello, Anonymous!
greet('John'); // Hello, John!
```

## Łańcuch opcjonalny

Operator łańcucha opcjonalnego `?.` działa jak zwykły operator kropki (`.`), służący do uzyskiwania dostępu do właściwości lub metod. Jednak bezpiecznie obsługuje wartości null i undefined, kończąc obliczanie wyrażenia i zwracając `undefined` zamiast zgłaszania błędu.

```
type Person = {
    name: string;
    age?: number;
    address?: {
        street?: string;
        city?: string;
    };
};

const person: Person = {
    name: 'John',
};

console.log(person.address?.city); // undefined
```

## Operator koalescencji null

Operator koalescencji null `??` zwraca wartość prawego operandu, jeśli lewy operand ma wartość `null` lub `undefined`; w

przeciwnym razie zwraca wartość lewego operandu.

```
const foo = null ?? 'foo';
console.log(foo); // foo

const baz = 1 ?? 'baz';
const baz2 = 0 ?? 'baz';
console.log(baz); // 1
console.log(baz2); // 0
```

## Typy literałów szablonowych

Typy literałów szablonowych pozwalają manipulować wartościami ciągów znaków na poziomie typów i generować nowe typy ciągów na podstawie istniejących. Są przydatne do tworzenia bardziej wyrazistych i precyzyjnych typów na podstawie operacji na ciągach znaków.

```
type Department = 'engineering' | 'hr';
type Language = 'english' | 'spanish';
type Id = `${Department}-${Language}-id`; // "engineering-english-id" | "engineering-spanish-id" | "hr-english-id" | "hr-spanish-id"
```

## Przeciążanie funkcji

Przeciążanie funkcji pozwala zdefiniować wiele sygnatur funkcji o tej samej nazwie, z których każda ma inne typy parametrów i typy zwracane. Gdy wywoływana jest przeciążona funkcja, TypeScript używa przekazanych argumentów do określenia właściwej sygnatury funkcji:

```
function makeGreeting(name: string): string;
function makeGreeting(names: string[]): string[];

function makeGreeting(person: unknown): unknown {
```

```
    if (typeof person === 'string') {
        return `Hi ${person}!`;
    } else if (Array.isArray(person)) {
        return person.map(name => `Hi, ${name}!`);
    }
    throw new Error('Unable to greet');
}

makeGreeting('Simon');
makeGreeting(['Simone', 'John']);
```

## Typy rekurencyjne

Typ rekurencyjny to typ, który może odwoływać się do samego siebie. Jest to przydatne podczas definiowania hierarchicznych lub rekurencyjnych struktur danych (potencjalnie zagnieżdżonych w nieskończoność), takich jak listy wiązane, drzewa i grafy.

```
type ListNode<T> = {
    data: T;
    next: ListNode<T> | undefined;
};
```

## Rekurencyjne typy warunkowe

W TypeScript można definiować złożone relacje między typami za pomocą logiki i rekurencji. Omówmy to w prosty sposób:

Typy warunkowe umożliwiają definiowanie typów na podstawie warunków logicznych:

```
type CheckNumber<T> = T extends number ? 'Number' : 'Not a number';
type A = CheckNumber<123>; // 'Number'
type B = CheckNumber<'abc'>; // 'Not a number'
```

Rekurencja oznacza, że typ odwołuje się do samego siebie we własnej definicji:

```
type Json = string | number | boolean | null | Json[] | { [key:
string]: Json };

const data: Json = {
    prop1: true,
    prop2: 'prop2',
    prop3: {
        prop4: [],
    },
};
```

Rekurencyjne typy warunkowe łączą logikę warunkową i rekurencję. Oznacza to, że definicja typu może zależeć od samej siebie poprzez logikę warunkową, tworząc złożone i elastyczne relacje między typami.

```
type Flatten<T> = T extends Array<infer U> ? Flatten<U> : T;

type NestedArray = [1, [2, [3, 4], 5], 6];
type FlattenedArray = Flatten<NestedArray>; // 2 | 3 | 4 | 5 |
1 | 6
```

## Obsługa modułów ECMAScript w Node

Node.js dodał obsługę modułów ECMAScript od wersji 15.3.0, a TypeScript obsługuje moduły ECMAScript dla Node.js od wersji 4.7. Obsługę tę można włączyć za pomocą właściwości `module` z wartością `nodenext` w pliku `tsconfig.json`. Oto przykład:

```
{
  "compilerOptions": {
    "module": "nodenext",
    "outDir": "./lib",
    "declaration": true
```

```
  }
}
```

Node.js obsługuje dwa rozszerzenia plików modułów: `.mjs` dla modułów ES i `.cjs` dla modułów CommonJS. Odpowiednie rozszerzenia plików w TypeScript to `.mts` dla modułów ES oraz `.cts` dla modułów CommonJS. Gdy kompilator TypeScript transpiluje te pliki do JavaScript, tworzy pliki `.mjs` i `.cjs`.

Aby używać modułów ES w projekcie, można ustawić właściwość `type` na `module` w pliku `package.json`. Informuje to środowisko Node.js, że projekt ma być traktowany jako projekt korzystający z modułów ES.

Ponadto TypeScript obsługuje także deklaracje typów w plikach `.d.ts`. Te pliki deklaracji dostarczają informacje o typach dla bibliotek lub modułów napisanych w TypeScript, umożliwiając innym programistom korzystanie z nich wraz ze sprawdzaniem typów i automatycznym uzupełnianiem w TypeScript.

## Funkcje asercji

W TypeScript funkcje asercji to funkcje, które poprzez swój typ zwracany wskazują, że określony warunek został zweryfikowany. W najprostszej postaci funkcja asercji sprawdza przekazany predykat i zgłasza błąd, gdy predykat przyjmuje wartość `false`.

```
function isNumber(value: unknown): asserts value is number {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
}
```

Można ją również zadeklarować jako wyrażenie funkcyjne:

```
type AssertIsNumber = (value: unknown) => asserts value is
number;
const isNumber: AssertIsNumber = value => {
    if (typeof value !== 'number') {
        throw new Error('Not a number');
    }
};
```

Funkcje asercji są podobne do strażników typów. Strażniki typów zostały pierwotnie wprowadzone w celu wykonywania kontroli w czasie działania i zapewnienia poprawnego typu wartości w określonym zakresie. Strażnik typu jest funkcją, która oblicza predykat typu i zwraca wartość logiczną wskazującą, czy predykat jest prawdziwy, czy fałszywy. Różni się to nieco od funkcji asercji, których celem jest zgłoszenie błędu zamiast zwrócenia `false`, gdy predykat nie jest spełniony.

Przykład strażnika typu:

```
const isNumber = (value: unknown): value is number => typeof
value === 'number';
```

## Wariadyczne typy krotek

Wariadyczne typy krotek to funkcja wprowadzona w TypeScript w wersji 4.0. Zacznijmy więc od przypomnienia, czym jest krotka:

Typ krotki to tablica o określonej długości, w której znany jest typ każdego elementu:

```
type Student = [string, number];
const [name, age]: Student = ['Simone', 20];
```

Termin „wariadyczny” oznacza nieokreśloną arność (przyjmowanie zmiennej liczby argumentów).

Krotka wariadyczna jest typem krotki mającym wszystkie dotychczasowe właściwości, ale jej dokładny kształt nie został jeszcze zdefiniowany:

```
type Bar<T extends unknown[]> = [boolean, ...T, number];

type A = Bar<[boolean]>; // [boolean, boolean, number]
type B = Bar<['a', 'b']>; // [boolean, 'a', 'b', number]
type C = Bar<[]>; // [boolean, number]
```

W powyższym kodzie widać, że kształt krotki jest definiowany przez przekazany typ generyczny `T`.

Krotki wariadyczne mogą przyjmować wiele typów generycznych, dzięki czemu są bardzo elastyczne:

```
type Bar<T extends unknown[], G extends unknown[]> = [...T, boolean, ...G];

type A = Bar<[number], [string]>; // [number, boolean, string]
type B = Bar<['a', 'b'], [boolean]>; // ["a", "b", boolean, boolean]
```

Dzięki nowym krotkom wariadycznym możemy korzystać z następujących możliwości:

- Elementy rozwijane w składni typu krotki mogą teraz być generyczne, dzięki czemu możemy reprezentować operacje wyższego rzędu na krotkach i tablicach, nawet gdy nie znamy rzeczywistych typów, na których działamy.
- Elementy reszty mogą występować w dowolnym miejscu krotki.

Przykład:

```
type Items = readonly unknown[];

function concat<T extends Items, U extends Items>(
    arr1: T,
    arr2: U
): [...T, ...U] {
    return [...arr1, ...arr2];
}

concat([1, 2, 3], ['4', '5', '6']); // [1, 2, 3, "4", "5", "6"]
```

## Typy opakowujące

Typy opakowujące odnoszą się do obiektów opakowujących, które służą do reprezentowania typów pierwotnych jako obiektów. Obiekty te zapewniają dodatkowe funkcje i metody niedostępne bezpośrednio dla wartości pierwotnych.

Podczas uzyskiwania dostępu do metody takiej jak `charAt` lub `normalize` na wartości pierwotnego typu `string` JavaScript opakowuje ją w obiekt `String`, wywołuje metodę, a następnie odrzuca ten obiekt.

Przykład:

```
const originalNormalize = String.prototype.normalize;
String.prototype.normalize = function () {
    console.log(this, typeof this);
    return originalNormalize.call(this);
};
console.log('\u0041'.normalize());
```

TypeScript odzwierciedla to rozróżnienie, udostępniając osobne typy dla wartości pierwotnych i odpowiadających im obiektów

opakowujących:

- string => String
- number => Number
- boolean => Boolean
- symbol => Symbol
- bigint => BigInt

Typy opakowujące zwykle nie są potrzebne. Należy unikać ich używania i zamiast tego korzystać z typów pierwotnych, na przykład `string` zamiast `String`.

## Kowariancja i kontrawariancja w TypeScript

Kowariancja i kontrawariancja opisują relacje między typami wewnątrz typów generycznych. `Dog` jest `Animal`, ale nie każdy `Animal` jest `Dog`.

**Kowariancja (zwracanie wartości):** funkcja zwracająca `Dog` może zastąpić funkcję zwracającą `Animal`. Każdy pies jest zwierzęciem, ale funkcja zwracająca dowolne zwierzę może zwrócić kota.

```
class Animal {
    name = '';
}

class Dog extends Animal {
    breed = '';
}

type Producer<T> = () => T;

declare let produceAnimal: Producer<Animal>;
declare let produceDog: Producer<Dog>;
```

```
produceAnimal = produceDog; // Valid
produceDog = produceAnimal; // Error
```

**Kontrawariancja (przyjmowanie wartości):** funkcja przyjmująca dowolne `Animal` może zastąpić funkcję przyjmującą `Dog`. Obsłuży również psy; funkcja przyjmująca tylko psy nie obsłuży bezpiecznie każdego zwierzęcia.

```
type Consumer<T> = (value: T) => void;

declare let consumeAnimal: Consumer<Animal>;
declare let consumeDog: Consumer<Dog>;

consumeDog = consumeAnimal; // Valid
consumeAnimal = consumeDog; // Error with strictFunctionTypes
```

Strzałki pokazują kierunek bezpiecznego przypisania:

- Dog -> Animal
- Producer<Dog> -> Producer<Animal> (covariance)
- Consumer<Animal> -> Consumer<Dog> (contravariance)

**Modyfikowalne tablice:** TypeScript pozwala także przypisać `Dog[]` do `Animal[]`. Obie zmienne wskazują tę samą tablicę; przez ogólniejszy typ można dodać `Animal`, które nie jest psem.

```
const dogs: Dog[] = [new Dog()];
const animals: Animal[] = dogs;

animals.push(new Animal()); // Allowed, but unsafe
// dogs now contains an Animal that is not a Dog.
```

**Opcja kompilatora:** przy włączonym `strictFunctionTypes` parametry samodzielnych funkcji są sprawdzane kontrawariantnie. Bez tej opcji TypeScript może dopuścić oba

kierunki (biwariancję), nawet gdy są niebezpieczne. Parametry deklaracji metod i konstruktorów stanowią wyjątek.

### Opcjonalne adnotacje wariancji parametrów typów

Od TypeScript 4.7 `out` oznacza kowariancję, `in` kontrawariancję, a `in out` inwariancję. TypeScript zwykle sam wywnioskuje wariancję. Adnotacje muszą odpowiadać użyciu parametru generycznego i nie zmieniają go dowolnie.

```
type Producer<out T> = () => T;
type Consumer<in T> = (value: T) => void;
type Transformer<in out T> = (value: T) => T;
```

## Sygnatury indeksowe ze wzorcami ciągów szablonowych

Sygnatury indeksowe ze wzorcami ciągów szablonowych pozwalają definiować elastyczne sygnatury indeksowe przy użyciu wzorców ciągów szablonowych. Ta funkcja umożliwia tworzenie obiektów, które można indeksować za pomocą określonych wzorców kluczy tekstowych, zapewniając większą kontrolę i precyzję podczas uzyskiwania dostępu do właściwości oraz manipulowania nimi.

Od wersji 4.4 TypeScript pozwala stosować sygnatury indeksowe dla symboli i wzorców ciągów szablonowych.

```
const uniqueSymbol = Symbol('description');

type MyKeys = `key-${string}`;

type MyObject = {
    [uniqueSymbol]: string;
```

```
    [key: MyKeys]: number;
};

const obj: MyObject = {
    [uniqueSymbol]: 'Unique symbol key',
    'key-a': 123,
    'key-b': 456,
};

console.log(obj[uniqueSymbol]); // Unique symbol key
console.log(obj['key-a']); // 123
console.log(obj['key-b']); // 456
```

## Operator satisfies

Operator `satisfies` pozwala sprawdzić, czy dany typ spełnia określony interfejs lub warunek. Innymi słowy, zapewnia, że typ ma wszystkie wymagane właściwości i metody konkretnego interfejsu. Jest to sposób na upewnienie się, że zmienna pasuje do definicji typu. Oto przykład:

```
type Columns = 'name' | 'nickName' | 'attributes';

type User = Record<Columns, string | string[] | undefined>;

// Type Annotation using `User`
const user: User = {
    name: 'Simone',
    nickName: undefined,
    attributes: ['dev', 'admin'],
};

// In the following lines, TypeScript won't be able to infer properly
user.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
```

```
user.nickName; // string | string[] | undefined

// Type assertion using `as`
const user2 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} as User;

// Here too, TypeScript won't be able to infer properly
user2.attributes?.map(console.log); // Property 'map' does not exist on type 'string | string[]'. Property 'map' does not exist on type 'string'.
user2.nickName; // string | string[] | undefined

// Using the `satisfies` operator we can properly infer the types now
const user3 = {
    name: 'Simon',
    nickName: undefined,
    attributes: ['dev', 'admin'],
} satisfies User;

user3.attributes?.map(console.log); // TypeScript infers correctly: string[]
user3.nickName; // TypeScript infers correctly: undefined
```

## Importy i eksporty wyłącznie typów

Importy i eksporty wyłącznie typów pozwalają importować lub eksportować typy bez importowania albo eksportowania wartości lub funkcji powiązanych z tymi typami. Może to być przydatne do zmniejszenia rozmiaru pakietu wynikowego.

Aby używać importów wyłącznie typów, można zastosować słowo kluczowe `import type`.

TypeScript pozwala używać rozszerzeń zarówno plików deklaracji, jak i implementacji (`.ts`, `.mts`, `.cts` oraz `.tsx`) w importach wyłącznie typów, niezależnie od ustawienia `allowImportingTsExtensions`.

Na przykład:

```
import type { House } from './house.ts';
```

Obsługiwane są następujące formy:

```
import type T from './mod';
import type { A, B } from './mod';
import type * as Types from './mod';
export type { T };
export type { T } from './mod';
```

## Deklaracja using i jawne zarządzanie zasobami

Deklaracja `using` jest niemutowalnym wiązaniem o zasięgu blokowym, podobnym do `const`, używanym do zarządzania zasobami wymagającymi zwolnienia. Po zainicjowaniu wiązania wartością metoda `Symbol.dispose` tej wartości jest rejestrowana, a następnie wykonywana przy opuszczaniu otaczającego zakresu blokowego.

Rozwiązanie to opiera się na mechanizmie Resource Management standardu ECMAScript, który jest przydatny do wykonywania niezbędnych zadań porządkowych po utworzeniu obiektu, takich jak zamykanie połączeń, usuwanie plików i zwalnianie pamięci.

Uwagi:

- Ze względu na niedawne wprowadzenie w TypeScript w wersji 5.2 większość środowisk wykonawczych nie obsługuje tej funkcji natywnie. Potrzebne będą polyfille dla: `Symbol.dispose`, `Symbol.asyncDispose`, `DisposableStack`, `AsyncDisposableStack`, `SuppressedError`.
- Ponadto należy skonfigurować plik `tsconfig.json` w następujący sposób:

```
{
    "compilerOptions": {
        "target": "es2022",
        "lib": ["es2022", "esnext.disposable", "dom"]
    }
}
```

Przykład:

```
//@ts-ignore
Symbol.dispose ??= Symbol('Symbol.dispose'); // Simple polyfill

const doWork = (): Disposable => {
    return {
        [Symbol.dispose]: () => {
            console.log('disposed');
        },
    };
};

console.log(1);

{
    using work = doWork(); // Resource is declared
    console.log(2);
} // Resource is disposed (e.g., `work[Symbol.dispose]()` is
evaluated)

console.log(3);
```

Kod wyświetli:

```
1
2
disposed
3
```

Zasób kwalifikujący się do zwolnienia musi być zgodny z interfejsem `Disposable`:

```
// lib.esnext.disposable.d.ts
interface Disposable {
    [Symbol.dispose](): void;
}
```

Deklaracje `using` rejestrują operacje zwalniania zasobów na stosie, zapewniając ich zwalnianie w kolejności odwrotnej do kolejności deklarowania:

```
{
    using j = getA(),
        y = getB();
    using k = getC();
} // disposes `C`, then `B`, then `A`.
```

Zasoby zostaną zwolnione nawet wtedy, gdy późniejszy kod spowoduje wystąpienie wyjątków. Może to spowodować, że zwalnianie zasobu zgłosi wyjątek, potencjalnie tłumiąc inny wyjątek. W celu zachowania informacji o stłumionych błędach wprowadzono nowy natywny wyjątek `SuppressedError`.

### Deklaracja await using

Deklaracja `await using` obsługuje zasób zwalniany asynchronicznie. Wartość musi mieć metodę

`Symbol.asyncDispose`, na którą oczekuje się na końcu bloku.

```
async function doWorkAsync() {
    await using work = doWorkAsync(); // Resource is declared
}     //     Resource     is     disposed     (e.g.,     `await
work[Symbol.asyncDispose]()` is evaluated)
```

Zasób zwalniany asynchronicznie musi być zgodny z interfejsem `Disposable` lub `AsyncDisposable`:

```
// lib.esnext.disposable.d.ts
interface AsyncDisposable {
    [Symbol.asyncDispose](): Promise<void>;
}

//@ts-ignore
Symbol.asyncDispose   ??=   Symbol('Symbol.asyncDispose');   //
Simple polyfill

class DatabaseConnection implements AsyncDisposable {
     // A method that is called when the object is disposed
asynchronously
    [Symbol.asyncDispose]() {
        // Close the connection and return a promise
        return this.close();
    }

    async close() {
        console.log('Closing the connection...');
             await  new  Promise(resolve  =>  setTimeout(resolve,
1000));
        console.log('Connection closed.');
    }
}

async function doWork() {
     // Create a new connection and dispose it asynchronously
when it goes out of scope
```

```
    await using connection = new DatabaseConnection(); // Resource is declared
    console.log('Doing some work...');
} // Resource is disposed (e.g., `await connection[Symbol.asyncDispose]()` is evaluated)

doWork();
```

Kod wyświetla:

```
Doing some work...
Closing the connection...
Connection closed.
```

Deklaracje `using` i `await using` są dozwolone w instrukcjach: `for`, `for-in`, `for-of`, `for-await-of`, `switch`.

## Atrybuty importu

Atrybuty importu w TypeScript 5.3 (etykiety importów) informują środowisko wykonawcze, jak obsługiwać moduły (JSON itp.). Zwiększa to bezpieczeństwo poprzez zapewnienie jednoznacznych importów i jest zgodne z Content Security Policy (CSP), umożliwiając bezpieczniejsze ładowanie zasobów. TypeScript zapewnia ich poprawność, ale interpretację dotyczącą obsługi konkretnych modułów pozostawia środowisku wykonawczemu.

Przykład:

```
import config from './config.json' with { type: 'json' };
```

Z importem dynamicznym:

```
const config = import('./config.json', { with: { type: 'json' } });
```

### Sprawdzanie składni wyrażeń regularnych

Od wersji 5.5.4 TypeScript sprawdza literały wyrażeń regularnych podczas kompilacji pod kątem typowych błędów (np. nieprawidłowej składni, błędnych odwołań wstecznych, funkcji nieobsługiwanych przez docelową wersję JavaScript). Pomaga to wcześniej wykrywać błędy, ale nie obejmuje ciągów przekazywanych do `new RegExp("...")`.

```
let r = /(a)\2/; // Error: This backreference refers to a group that does not exist.
```

## import defer

`import defer` pozwala załadować moduł, ale opóźnić jego wykonanie do chwili rzeczywistego użycia czegoś z tego modułu. Pomaga to uniknąć niepotrzebnej pracy i efektów ubocznych.

- Działa tylko z: `import defer * as name from "module"`.
- Kod jest wykonywany dopiero po uzyskaniu dostępu do eksportowanego elementu.